

LEAGUE PILOT

Sistem software pentru planificarea și reconfigurarea competițiilor sportive

Candidat:

Laurențiu-Ștefan, Paul

Profesor coordonator:

Prof. dr. ing. Valer, Bocan

Sesiunea: Iulie 2026

REZUMAT

Organizarea și administrarea competițiilor sportive reprezintă un proces complex ce este influențat de diferite constrângeri logistice, temporale și organizaționale. Realizarea unui calendar competițional valid în orice sport necesită din partea organizatorilor o atenție sporită deoarece se iau în considerare mai multe aspecte, cum ar fi disponibilitatea echipelor participante, a infrastructurii sportive și a structurii competiției, iar în acest context, apariția unor evenimente neprevăzute poate impune modificarea planificării inițiale și reconfigurarea întregului sezon competițional.

Această lucrare propune dezvoltarea unui sistem software ce are ca scop planificarea și reconfigurarea competițiilor sportive. Soluția implementată permite atât generarea automată a unui calendar competițional cât și adaptarea acestuia în funcție de evenimentele dinamice ce îi pot afecta desfășurarea în parametri optimi. Acest sistem este bazat pe constrângeri pentru identificarea unor intervale valide de programare și pentru menținerea consistenței calendarului în urma modificărilor apărute pe parcursul unui sezon competițional.

Pentru a putea demonstra și valida funcționalitățile dezvoltate s-a optat pentru folosirea baschetului ca studiu de caz. Mai exact, s-a utilizat modelul competițional al baschetului profesionist din România. Această aplicație implementează după modelul din realitate fiecare stadiu dintr-un sezon: Supercupa României, sezon regulat, Cupa României și fazele play-off / play-out după sistemul elvețian. În vederea simulării unui scenariu cât mai aproape de realitate, aplicația prezintă și componente pentru gestionarea echipelor participante în sezonul 2025/2026 în România, dar și jucătorii fiecărei participante în sezon. Pe lângă acestea, aplicația are încorporat atât clasarea finală pentru fiecare stagi, cât și rezultatele înregistrate.

Această opțiune de a simula un sezon oferă un context concret pentru evaluarea mecanismelor de planificare și reconfigurare. În urma unei astfel de simulări este posibilă analizarea comportamentului sistemului în prezența unor situații ce impun modificarea calendarului și reorganizarea desfășurării competiției.

Din punct de vedere tehnic, soluția a fost implementată utilizând o arhitectură client-server bazată pe FastAPI, React și PostgreSQL, fiind

distribuită prin intermediul containerelor Docker. Rezultatele obținute demonstrează capacitatea sistemului de a genera și administra competiții sportive complexe, reducând efortul necesar ce trebuie acordat procesului de organizare și oferind mecanisme flexibile de adaptare la schimbările apărute pe parcursul desfășurării unui sezon.

CUPRINS

1. INTRODUCERE	- 6 -
1.1 CONTEXT	- 6 -
1.2 MOTIVAREA ALEGERII TEMEI	- 7 -
1.3 PROBLEMA ABORDATĂ	- 8 -
1.4 OBIECTIVELE LUCRĂRII	- 9 -
1.5 STRUCTURA LUCRĂRII	- 10 -
2. FUNDAMENTE TEORETICE	- 12 -
2.1 ORGANIZAREA COMPETIȚIILOR SPORTIVE	- 12 -
2.2 STRUCTURA COMPETIȚIILOR DE BASCHET	- 13 -
2.3 PROBLEMELE DE PLANIFICARE (SCHEDULING)	- 15 -
2.4 CONSTRÂNGERI ÎN PROCESUL DE PLANIFICARE	- 16 -
2.5 REPROGRAMAREA ȘI GESTIONAREA EVENIMENTELOR DINAMICE	- 18 -
2.6 SIMULAREA COMPETIȚIILOR SPORTIVE	- 21 -
3. ANALIZA ȘI PROIECTAREA SISTEMULUI	- 24 -
3.1 CERINȚELE FUNCȚIONALE	- 24 -
3.2 CERINȚELE NEFUNCȚIONALE	- 26 -
3.3 ACTORII SISTEMULUI	- 27 -
3.4 CAZURI DE UTILIZARE	- 29 -
3.5 PROIECTAREA CONCEPTUALĂ A SISTEMULUI	- 31 -
3.6 COMPONENTELE PRINCIPALE ALE SISTEMULUI	- 33 -
4. ARHITECTURA ȘI TEHNOLOGIILE UTILIZATE	- 35 -
4.1 ARHITECTURA SOFTWARE	- 35 -
4.2 COMPONENTA FRONTEND	- 38 -
4.3 COMPONENTA BACKEND	- 41 -

4.4	MODELUL DE DOMENIU	- 43 -
4.5	PERSISTENȚA DATELOR	- 44 -
4.6	CONTAINERIZAREA APLICAȚIEI	- 46 -
4.6.1	MODURI DE DISTRIBUIRE ȘI ACCES	- 49 -
5.	IMPLEMENTAREA SISTEMULUI	- 51 -
5.1	STRUCTURA PROIECTULUI	- 51 -
5.2	IMPLEMENTAREA MODELULUI DE DOMENIU	- 52 -
5.2.1	ENTITATEA TEAM	- 52 -
5.2.2	ENTITATEA PLAYER	- 53 -
5.2.3	ENTITATEA GAME	- 54 -
5.3	IMPLEMENTAREA MECANISMULUI DE GENERARE A CALENDARULUI COMPETIȚIONAL	- 55 -
5.3.1	GENERAREA MECIURILOR	- 55 -
5.3.2	ALOCAREA MECIURILOR ÎN CALENDAR	- 56 -
5.4	IMPLEMENTAREA MECANISMULUI DE SIMULARE A SEZONULUI	- 58 -
5.5	IMPLEMENTAREA MECANISMULUI DE REPROGRAMARE	- 60 -
5.6	IMPLEMENTAREA GESTIONĂRII EVENIMENTELOR DINAMICE	- 61 -
5.7	IMPLEMENTAREA API-ULUI	- 62 -
5.8	IMPLEMENTAREA INTERFEȚEI UTILIZATOR	- 64 -
6.	TESTAREA ȘI VALIDAREA SISTEMULUI	- 67 -
6.1	METODOLOGIA DE TESTARE	- 67 -
6.2	VALIDAREA GENERĂRII CALENDARULUI	- 67 -
6.3	VALIDAREA SIMULĂRII SEZONULUI	- 68 -
6.4	VALIDAREA MECANISMULUI DE REPROGRAMARE	- 69 -
6.5	ANALIZA REZULTATELOR OBȚINUTE	- 70 -
7.	CONCLUZII ȘI DIRECȚII VIITOARE DE DEZVOLTARE	- 71 -

1. INTRODUCERE

1.1 CONTEXT

Competițiile sportive presupun în general gestionarea unui număr ridicat de participanți, resurse sau constrângeri organizaționale. Pe lângă desfășurarea în sine a evenimentelor, organizatorii trebuie să se asigure de planificarea partidelor că este una eficientă, de utilizarea corespunzătoare a infrastructurii disponibile și de respectarea regulamentelor specifice competiției. Astfel, realizarea unui calendar competițional reprezintă una dintre cele mai importante activități necesare desfășurării unui sezon.

Tot acest proces de planificare a competiției este unul amplu ce nu se rezumă doar la stabilirea meciurilor între participantele din competiție. Acesta ține cont simultan și de alți factori precum: disponibilitatea cluburilor și a bazelor sportive, intervalele dintre două partide, structura competiției respective, dar și a celor ce se desfășoară în paralel, mai ales dacă vorbim de competiții organizate de o federație, fie ea a unei țări sau de cea internațională sau de cele private. Pe măsură ce numărul constrângerilor crește, identificarea unei planificări valide devine din ce în ce mai dificilă.

Practic, dificultatea procesului de organizare crește doar în momentul în care apar evenimente ce nu pot fi anticipate în momentul generării calendarului inițial. Participarea unor echipe în competiții suplimentare, indisponibilitatea temporară a unor arene, diferitele modificări logistice sau apariția altor situații excepționale pot conduce la amânarea și reprogramarea unor meciuri deja stabilite cu mult timp înainte. Apariția unor astfel de evenimente produc frecvent efecte în lanț asupra întregului calendar competițional și pot necesita revizuirea unei părți semnificative a planificării existente.

Putem afirma faptul că aceste probleme fac parte din categoria celor de planificare și alocare a resurselor. Crearea unor astfel de sisteme au ca scop identificarea unei configurații care respectă simultan setul de constrângeri și care poate fi modificată în orice moment dacă apar schimbări ale condițiilor inițiale. Acest tip de probleme nu este întâlnit doar în sport. Îl regăsim și în alte domenii, cum ar fi transportul, logistica, organizarea de

evenimente (festivaluri, concerte, etc.), unde planificarea și replanificarea activităților reprezintă procese esențiale.[5][17]

Dezvoltarea unor aplicații software dedicate automatizării proceselor de planificare a devenit o direcție importantă și în mediul academic, nu doar în cel profesional. Utilizarea unor astfel de sisteme capabile să genereze și să adapteze automat calendare complexe contribuie la reducerea efortului administrativ, diminuarea erorilor umane și la creșterea eficienței procesului decizional.

Astfel, lucrarea propune dezvoltarea unui sistem software capabil să planifice și să reconfigureze o competiție sportivă în mod automat, utilizând ca model o competiție de baschet profesionist, mai exact cea din România. Soluția urmărește să ofere un mecanism flexibil de gestionare a calendarului competițional și să demonstreze modul în care un eveniment sau mai multe neprevăzute pot fi integrate fără a compromite buna desfășurare a sezonului.

1.2 MOTIVAREA ALEGERII TEMEI

Decizia de a alege această temă a fost din dorința mea proprie de a realiza ceva diferit, la care s-a adăugat ampla analiză asupra situației din sportul românesc, în special în urma observării unor dificultăți întâlnite în organizarea competițiilor din baschetul profesionist din România. În ultimii ani, competiția națională a avut de suferit din diverse cauze, printre exemple numărându-se oprirea sa în mijlocul sezonului în pandemia de COVID-19, nerezultând o campioană, fiind printre puținele campionate în care nu s-au găsit soluții pentru finalizarea sezonului. Un alt exemplu este foarte recent, din sezonul proaspăt încheiat, în care din cauza unei competiții ce a optat pentru competiții internaționale mai puternice, dar și private în același timp, nu s-a reușit să se găsească soluții în programarea etapelor pentru a putea lua parte și în întrecerea națională, astfel că decizia federației a fost de a nu lua parte la sezonul regulat, doar din stagiul de Play-off, fiind practic direct calificată în această fază, spre deosebire de restul formațiilor care au fost nevoite să lupte un sezon întreg pentru a obține o poziție în Play-off. Ajunși în Play-off, din cauza participării în paralel și în competiția privată, faza semifinalelor a fost amânată de două ori, fără a avea un răspuns din partea organizatorilor cu privire la reprogramare, așteptând peste 48 de ore pentru o decizie în fapt banală, cu atât mai mult cu cât cealaltă semifinală deja începuse. În timpul sezonului regulat au existat de asemenea

suficiente meciuri amânate la o distanță de câteva luni, nereușindu-se o programare optimă care să nu încurce restul participantelor la competiție, organizatorii nedisponând de suficiente informații pentru a stabili cu exactitate desfășurarea etapelor următoare.

În astfel de situații, o simplă modificare a unei partide poate influența disponibilitatea tuturor, de la echipe la jucători ce pot fi accidentați la reprogramare, la posibile indisponibilități ale sălilor de joc din cauza unor alte evenimente programate pentru aceea perioadă a anului sau alte evenimente ce îngreunează buna desfășurare a competiției.

Pornind de la această problemă reală, a apărut ideea dezvoltării unui sistem software capabil să genereze și să reconfigureze automat calendarul unei competiții sportive. Scopul principal este acela de reducere a timpului necesar procesului de planificare și identificare rapidă a unor alternative viabile atunci când apar evenimente care afectează desfășurarea normală a sezonului.

Pentru a valida soluția propusă a fost ales baschetul drept studiu de caz. Acest domeniu oferă un mediu potrivit pentru analiza problemelor de planificare datorată structurii complexe a competițiilor, existența mai multor faze competiționale și posibilitatea apariției unor situații care necesită modificări ale calendarului inițial. Totodată, utilizarea unui scenariu apropiat de realitate facilitează evaluarea comportamentului sistemului și evidențierea avantajelor oferite de automatizarea procesului de organizare.

1.3 PROBLEMA ABORDATĂ

Problema principală abordată în cadrul acestei lucrări este generarea și reconfigurarea unui calendar competițional în prezența unor constrângeri și a unor evenimente care pot modifica desfășurarea planificată a competiției.

La cea mai simplă formă a sa, procesul de planificare presupune stabilirea unui calendar care să permită disputarea tuturor meciurilor necesare unei competiții. În practică, acest lucru este influențat de numeroși factori care trebuie respectați simultan. Disponibilitatea echipelor participante sau disponibilitatea arenelor, structura competiției reprezintă doar câteva dintre constrângerile care trebuie luate în considerare.[17]

Dificultatea problemei dezbătute în lucrare crește odată cu apariția unor evenimente neprevăzute care afectează calendarul inițial. În cazul unor astfel de situații, simpla amânare a unui meci nu este suficientă, fiind

necesară identificarea unor noi intervale disponibile și verificarea impactului asupra celorlalte partide deja programate. O singură modificare poate genera efecte în lanț asupra întregii competiții, ceea ce transformă reprogramarea într-un proces complex.[9]

Această lucrare se bazează pe o abordare ce automatizează acest proces. Sistemul prezentat urmărește generarea unui calendar valid și adaptarea acestuia atunci când apar situații care impun modificări ale planificării inițiale. În acest mod, soluția reduce intervenția manuală și facilitează administrarea competițiilor sportive desfășurate pe perioade lungi de timp.

Chiar dacă această implementare este bazată pe competiția de baschet din România drept studiu de caz, problema abordată este una generală ce poate fi întâlnită și în alte domenii în care este necesară planificarea și replanificarea evenimentelor în funcție de constrângeri și de schimbările apărute pe parcursul desfășurării acestora.

1.4 OBIECTIVELE LUCRĂRII

Obiectivul principal al lucrării este dezvoltarea unui sistem software capabil să genereze și să reconfigureze calendarul competițional pentru competițiile sportive desfășurate pe parcursul unui sezon în România. Soluția propusă urmărește automatizarea procesului de planificare și oferirea unor mecanisme care permit adaptarea calendarului atunci când apar evenimente care afectează desfășurarea normală a competiției.

Un prim obiectiv al sistemului este reprezentat de modelarea structurii unei competiții sportive. Pentru validarea funcționalităților au fost implementate principalele etape ale unui sezon competițional: Supercupa României, sezonul regulat, Cupa României, dar și fazele de play-off și play-out.

Un alt obiectiv este constituit de generarea automată a calendarului competițional. Sistemul va programa meciurile respectând constrângeri definite și disponibilitatea resurselor implicate în desfășurarea competiției.

Lucrarea urmărește dezvoltarea unor mecanisme de reconfigurare a calendarului. În situațiile în care apar evenimente care afectează planificarea inițială, sistemul trebuie să poată identifica soluții alternative pentru a menține

competiția stabilă, fără a fi nevoie de intervenții majore din partea utilizatorului.

Pentru evaluarea comportamentului acestui sistem a fost integrat și un modul de simulare a unui sezon competițional. Acesta permite generarea rezultatelor meciurilor, actualizarea clasamentelor și determinarea evoluției competiției pe parcursul întregului sezon. Scopul acestui modul este de a furniza un mediu realist pentru testarea și validarea mecanismelor de planificare și reconfigurare.

Un ultim obiectiv este reprezentat de dezvoltarea unei interfețe web care să permită configurarea simulărilor, monitorizarea desfășurării competiției și vizualizarea rezultatelor generate. Prin intermediul acestei interfețe utilizatorul poate interacționa cu sistemul într-un mod intuitiv și poate analiza informațiile produse în urma simulărilor efectuate.

Metodologia utilizată în cadrul acestei lucrări presupune analiza problemelor de planificare competițională, identificarea constrângerilor specifice, proiectarea unei arhitecturi software adecvate și implementarea unui sistem capabil să genereze, simuleze și reconfigureze automat un calendar competițional.

Elementul de noutate al lucrării constă în integrarea într-un singur sistem software a mecanismelor de planificare, simulare și reprogramare a competițiilor sportive, adaptate unui model real din baschetul profesionist românesc.

1.5 STRUCTURA LUCRĂRII

Lucrarea este organizată pe mai multe capitole, fiecare abordând o etapă distinctă a procesului de analiză, proiectare, implementare și evaluare a sistemului dezvoltat.

Capitolul 1 prezintă contextul lucrării, motivația alegerii temei, problema abordată și obiectivele urmărite pe parcursul dezvoltării proiectului.

Capitolul 2 introduce fundamentele teoretice necesare înțelegerii problemei de planificare și reconfigurare a competițiilor sportive. În cadrul acestuia sunt prezentate concepte legate de organizarea competițiilor sportive, constrângerile care influențează procesul de planificare și mecanismele de reprogramare utilizate în situații dinamice.

Capitolul 3 descrie analiza și proiectarea sistemului, fiind prezentate cerințele funcționale și nefuncționale, principalele componente ale aplicației

și modul în care acestea interacționează pentru realizarea funcționalităților propuse.

Capitolul 4 este dedicat arhitecturii aplicației și tehnologiilor utilizate în implementare. Sunt prezentate structura generală a sistemului, componentele backend și frontend, dar și soluțiile utilizate pentru persistarea și gestionarea datelor.

Capitolul 5 tratează aspectele de implementare, evidențiind principalele mecanisme dezvoltate în cadrul aplicației. În această secțiune sunt prezentate componentele responsabile de generarea calendarului competițional, gestionarea evenimentelor și simularea desfășurării sezonului.

Capitolul 6 prezintă modul de utilizare al aplicației, procesul de testare, simulare și evaluare a rezultatelor obținute dar și validarea principalelor funcționalități disponibile utilizatorului, fiind ilustrate scenarii de utilizare și exemple de rezultate generate de sistem.

Capitolul 7 conține concluziile lucrării, principalele contribuții ale proiectului și posibile direcții de dezvoltare ulterioară.

2. FUNDAMENTE TEORETICE

2.1 ORGANIZAREA COMPETIȚIILOR SPORTIVE

Competițiile sportive sunt structuri ce au rolul de a desfășura întreceri între participanți în condiții prestabilite și conform unor regulamente specifice. În cadrul lor, echipele concurează pe parcursul unei perioade determinate de timp, iar rezultatele obținute sunt utilizate pentru a stabili un clasament și câștigătorul competiției.[7]

Din punct de vedere organizatoric, desfășurarea unei competiții sportive implică un număr ridicat de activități ce trebuie să fie coordonate simultan. Printre acestea se numără stabilirea unui format competițional, programarea partidelor, gestionarea infrastructurii sportive, coordonarea participanților și respectarea regulamentelor impuse de organizatorii acestora. Complexitatea acestor activități crește odată cu numărul de participanți și cu durata competiției.[8]

Un element esențial pentru o astfel de organizare este reprezentat de calendarul competițional. Acesta stabilește momentul desfășurării fiecărei partide și definește ordinea în care sunt disputate etapele competiției. Un calendar bine definit contribuie la desfășurarea eficientă a sezonului și la evitarea conflictelor care pot apărea între diferitele activități asociate unei competiții.[17]

Procesul de planificare este influențat de numeroși factori: disponibilitatea echipelor participante, disponibilitatea sălilor de joc, intervalele necesare pentru recuperarea sportivilor și structura competiției. Acestea sunt doar câteva dintre aspectele ce trebuie să le luăm în considerare în procesul de organizare. În plus, apariția unor situații neprevăzute poate impune modificarea planificării inițiale și adaptarea calendarului competițional.[17][9]

Dezvoltarea tehnologiilor software a permis automatizarea unei părți importante a proceselor de organizare și administrare a competițiilor sportive. Utilizarea unor sisteme informatice specializate contribuie la reducerea timpului necesar planificării, la reducerea erorilor produse de om și la

identificarea rapidă a unor soluții alternative în situațiile care necesită modificarea calendarului competițional.[5]

2.2 STRUCTURA COMPETIȚIILOR DE BASCHET

Baschetul utilizează diferite formate competiționale în funcție de regulamentul și obiectivele fiecărei competiții. În general, un sezon este împărțit în mai multe etape, fiecare având un rol specific în determinarea clasamentului final și a câștigătoareii.

Prima etapă o reprezintă sezonul regulat. Aici, echipele participante dispută un număr prestabilit de meciuri, iar rezultatele obținute sunt utilizate pentru stabilirea unui clasament general. Acest clasament determină ulterior poziționarea echipelor în fazele eliminatorii sau în alte etape ale competiției.[7]

Pe lângă, mai sunt incluse și cupele naționale și supercupele. Acestea se pot desfășura după reguli diferite față de campionat și contribuie la creșterea complexității procesului de organizare. Integrarea acestora în calendarul competițional necesită o planificare mai atentă pentru a evita suprapunerea cu celelalte evenimente sportive.[8]

Mai avem fazele eliminatorii, ce reprezintă o componentă importantă a competițiilor de baschet. În cadrul acestora, echipele sunt împerecheate pe baza poziției ocupate în clasamentul sezonului regulat, iar câștigătoarele avansează în runda următoare. Acest sistem permite stabilirea unei ierarhii finale și oferă un nivel ridicat de competitivitate în partea finală a sezonului.

În cadrul unei competiții, de regulă există și faze dedicate echipelor care nu se califică în lupta pentru titlu (denumirea populară de play-out) sau faze pentru cele eliminate din lupta pentru titlu (de exemplu clasarea pentru locurile 5-8, finala mică). Aceste etape permit stabilirea pozițiilor finale din clasament și contribuie la obținerea unei imagini complete asupra performanței tuturor participanților la competiție.[7][8]

Complexitatea unui sezon este determinată de numărul de echipe participante, dar și de diversitatea etapelor care trebuie integrate într-un singur calendar. Din acest motiv, baschetul reprezintă un studiu de caz potrivit pentru analiza problemelor de planificare și reconfigurare abordate în cadrul acestei lucrări.

Pentru a putea valida sistemul dezvoltat a fost utilizată o structură competițională inspirată din organizarea baschetului profesionist din România. Modelul implementat include Supercupa României, sezonul

regulat, Cupa României și fazele de play-off și play-out, oferind un mediu suficient de complex pentru evaluarea mecanismelor de generare și adaptare a calendarului competițional.

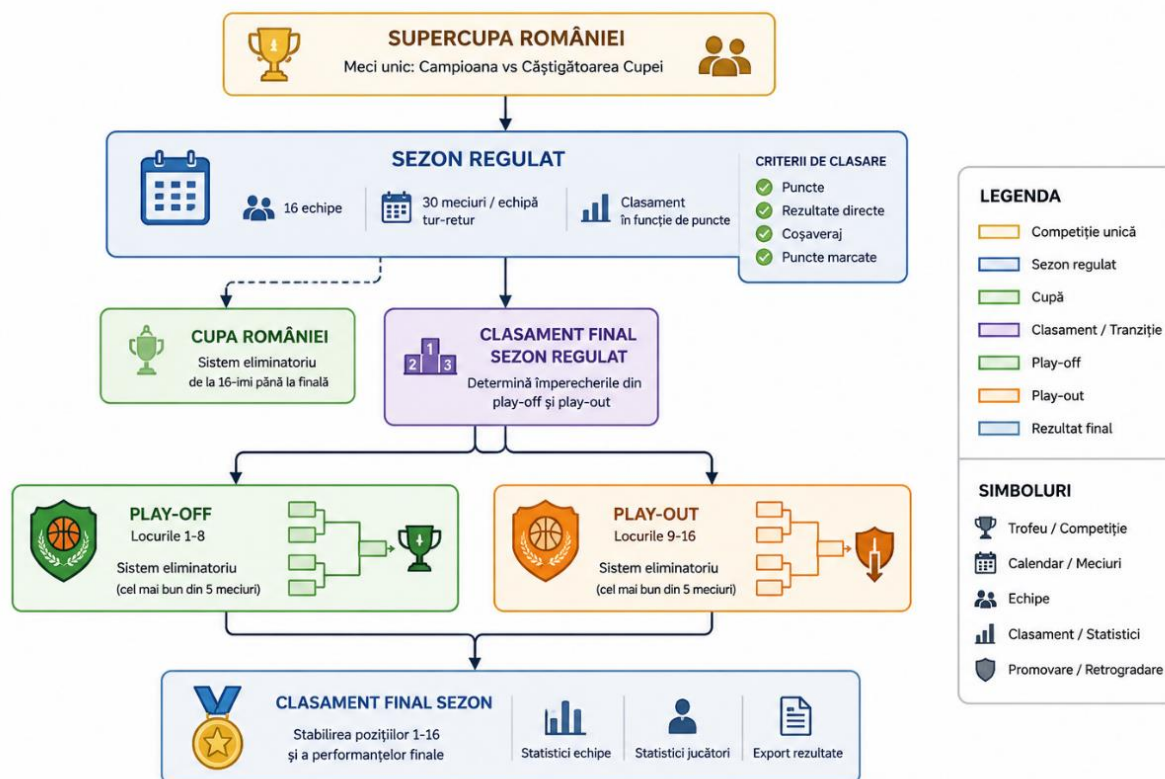


Figura 2.1 – Structura sezonului competițional utilizat în aplicație, adaptare proprie după regulamentul FRB[8]

Figura 2.1 ilustrează fluxul general al sezonului competițional utilizat în cadrul aplicației. Competiția debutează cu Supercupa României, urmată de desfășurarea sezonului regulat și a Cupei României. Clasamentul obținut la finalul sezonului regulat determină împărțirea echipelor în fazele de play-off și play-out. Rezultatele obținute în aceste etape conduc la stabilirea clasamentului final al sezonului, precum și la generarea statisticilor și a informațiilor utilizate în cadrul simulării.

2.3 PROBLEMELE DE PLANIFICARE (SCHEDULING)

Planificarea activităților reprezintă una dintre cele mai importante probleme întâlnite în domeniul informaticii și al cercetării operaționale. În

forma sa generală, o problemă de planificare presupune alocarea unor activități într-un interval de timp astfel încât să fie respectat un set de constrângeri și să fie atinse anumite obiective prestabilite.[17]

Problemele de planificare sunt întâlnite într-o varietate de domenii, precum transportul, logistica, producția industrială, managementul proiectelor și organizarea evenimentelor. În toate aceste situații există un număr limitat de resurse disponibile, iar activitățile trebuie programate astfel încât să nu apară conflicte sau suprapuneri.[9]

În cazul competițiilor sportive, procesul de planificare presupune stabilirea momentului de desfășurare pentru fiecare meci din cadrul sezonului competițional. Obiectivul principal este generarea unui calendar valid care să permită desfășurarea tuturor partidelor necesare și care să respecte regulile competiției.[16]

Complexitatea unei astfel de probleme crește odată cu numărul participanților și cu numărul constrângerilor care trebuie respectate simultan. Disponibilitatea echipelor, disponibilitatea sălilor de joc, intervalele de recuperare dintre meciuri și structura diferitelor faze competiționale reprezintă câteva exemple de factori care influențează procesul de planificare.[11]

În multe situații nu există o singură soluție posibilă, ci un număr mare de variante valide. Din acest motiv, sistemele software moderne utilizează algoritmi și mecanisme specializate pentru identificarea unor planificări care respectă constrângerile definite și care pot fi adaptate atunci când apar modificări ale condițiilor inițiale.[23]

Problema abordată în cadrul acestei lucrări poate fi încadrată în această categorie de probleme de planificare. Sistemul dezvoltat urmărește generarea unui calendar competițional valid și identificarea unor soluții alternative atunci când apar evenimente care afectează programarea inițială a meciurilor.

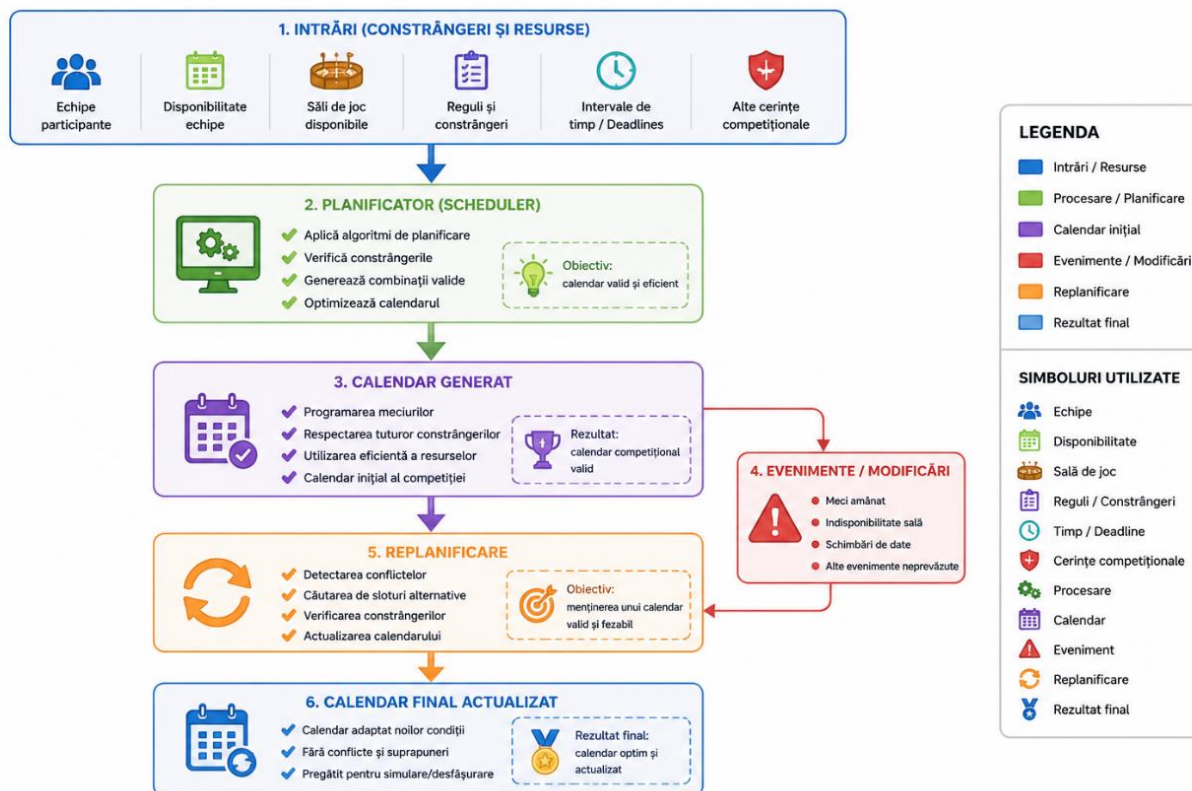


Figura 2.2 - Model general al unei probleme de planificare (scheduling)

Figura 2.2 prezintă modelul general al unei probleme de planificare. Procesul pornește de la un set de intrări și constrângeri, precum participanții implicați, resursele disponibile și regulile competiției. Aceste informații sunt utilizate de mecanismul de planificare pentru generarea unui calendar inițial care respectă condițiile definite. În situația apariției unor evenimente neprevăzute sau a unor modificări ale condițiilor inițiale, este necesară reluarea procesului de analiză și identificarea unor alternative valide. Rezultatul final îl reprezintă un calendar actualizat, capabil să respecte atât constrângerile inițiale, cât și modificările apărute pe parcursul desfășurării competiției.

2.4 CONSTRÂNGERI ÎN PROCESUL DE PLANIFICARE

În orice problemă de planificare, constrângerile reprezintă condițiile care trebuie respectate pentru ca soluția obținută să fie considerată validă.

Acestea limitează numărul variantelor posibile și influențează direct modul în care activitățile pot fi programate într-un interval de timp determinat.[9]

În cadrul competițiilor sportive, constrângerile apar din mai multe surse, unele fiind determinate de regulamentele competiției, iar altele de disponibilitatea resurselor implicate în desfășurarea acesteia. Pentru ca un calendar competițional să poate fi pus în practică, toate aceste condiții trebuie respectate simultan.[9]

O categorie importantă de constrângeri este reprezentată de disponibilitatea participanților. O echipă nu poate fi programată să dispute două meciuri în același interval de timp și trebuie să beneficieze de perioade rezonabile de recuperare între partide. Nerespectarea acestor condiții poate afecta atât desfășurarea competiției, cât și performanța participanților.[9]

O altă categorie este reprezentată de disponibilitatea infrastructurii sportive. Sălile de joc și terenurile pentru desfășurarea meciurilor pot avea intervale limitate de utilizare sau pot fi indisponibile în anumite perioade. Din acest motiv, procesul de planificare trebuie să țină cont de resursele disponibile în momentul programării unei partide.[9]

Structura competiției introduce și ea constrângeri suplimentare. În majoritatea competițiilor sportive există o ordine clară în care trebuie disputate etapele, iar anumite meciuri nu pot fi programate înainte de finalizarea altora. Aceste dependențe trebuie respectate pentru a asigura desfășurarea corectă a competiției.[9]

Pe lângă constrângerile inițiale, procesul de organizare poate fi influențat și de apariția unor evenimente neprevăzute. Amânarea unui meci, indisponibilitatea unei resurse sau modificarea unor condiții existente poate conduce la apariția unor conflicte de programare. În astfel de situații, sistemul trebuie să identifice soluții alternative care să permită menținerea unui calendar valid.

Din perspectiva sistemelor software, gestionarea eficientă a constrângerilor reprezintă unul dintre elementele esențiale ale procesului de planificare. Capacitatea de a verifica respectarea condițiilor impuse și de a adapta calendarul atunci când apar modificări constituie baza funcționării aplicațiilor moderne de scheduling și rescheduling.

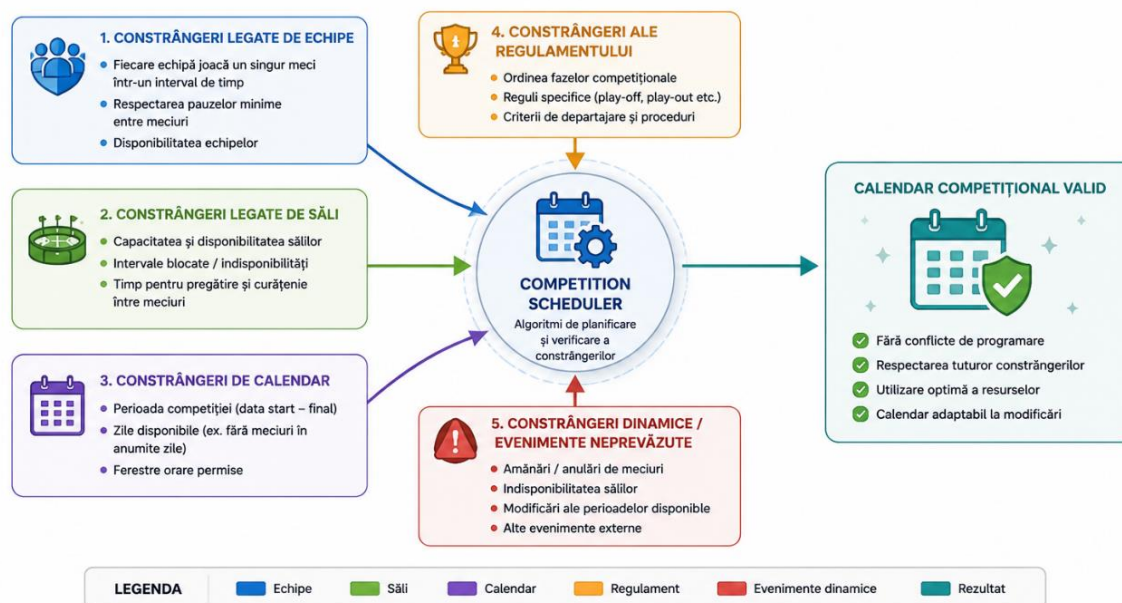


Figura 2.3 - Principalele categorii de constrângeri utilizate în planificare

Figura 2.3 ilustrează principalele categorii de constrângeri care influențează procesul de planificare a competițiilor de baschet. Constrângerile sunt provenite din caracteristicile echipelor participante, din disponibilitatea infrastructurii, din structura calendarului, din regulamentele competiției și din evenimentele dinamice care pot apărea pe parcurs. Mecanismul de planificare trebuie să integreze simultan toate aceste condiții pentru a genera un calendar valid și fezabil.

2.5 REPROGRAMAREA ȘI GESTIONAREA EVENIMENTELOR DINAMICE

În majoritatea competițiilor sportive, calendarul competițional este stabilit înainte de începerea sezonului și reprezintă baza organizării tuturor activităților ulterioare. În practică, deseori apar situații ce pot afecta planificarea inițială și care impun modificarea programului competițional.[17]

Astfel de situații sunt denumite evenimente dinamice și pot avea multiple cauze. Printre acestea se numără indisponibilitatea infrastructurii sportive, modificarea perioadelor disponibile pentru desfășurarea meciurilor, apariția unor conflicte de programare sau alte evenimente care împiedică disputarea unei partide la data stabilită inițial.[9]

În prezența unor astfel de evenimente, simpla eliminare sau amânare a unui meci nu reprezintă o soluție suficientă. Modificarea unei singure partide poate influența desfășurarea etapelor următoare și poate genera conflicte suplimentare în calendarul competițional. Din acest motiv, este necesar un proces de reprogramare care să permită identificarea unor noi intervale disponibile și integrarea acestora în planificarea existentă.

Procesul de reprogramare presupune detectarea conflictului apărut, analiza impactului asupra calendarului existent și identificarea unor alternative care respectă constrângerile definite. În funcție de complexitatea competiției, acest proces poate deveni dificil atunci când numărul meciurilor și al restricțiilor crește.[17]

Sistemele moderne de planificare utilizează mecanisme automate pentru gestionarea acestor situații. Prin verificarea constrângerilor și evaluarea variantelor disponibile, acestea pot identifica rapid soluții care permit continuarea competiției fără afectarea semnificativă a calendarului general.[11]

În cadrul lucrării, mecanismele de reprogramare reprezintă una dintre componentele principale ale sistemului dezvoltat. Acestea permit adaptarea calendarului competițional atunci când apar evenimente care afectează desfășurarea normală a sezonului, contribuind la menținerea unei planificări valide și coerente pe întreaga durată a competiției.

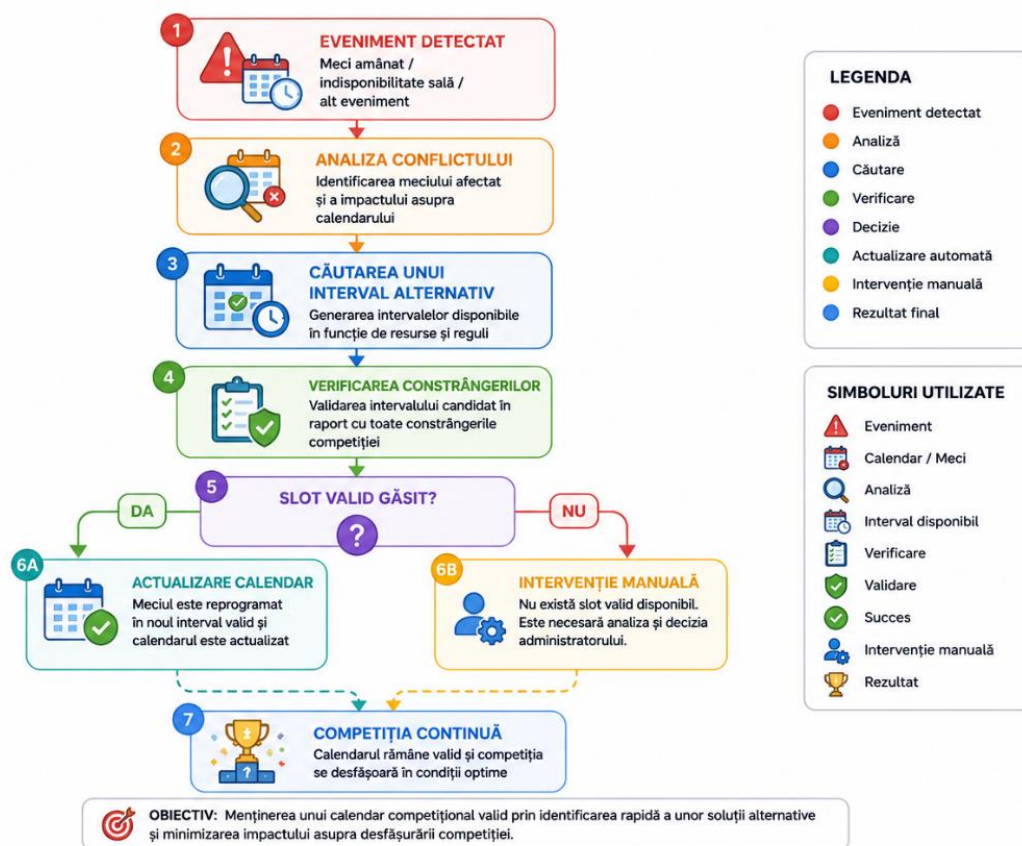


Figura 2.4 - Procesul general de reprogramare a unui meci

Figura 2.4 prezintă procesul general de reprogramare a unui meci în urma apariției unui eveniment care afectează calendarul competițional. Procesul începe prin detectarea unei situații care împiedică desfășurarea partidei conform planificării inițiale, cum ar fi indisponibilitatea unei săli de joc, amânarea unui meci sau apariția unui conflict de programare.

După identificarea problemei, sistemul analizează impactul acesteia asupra calendarului existent și inițiază procesul de căutare a unor intervale alternative disponibile. Fiecare variantă identificată este verificată în raport cu constrângerile competiției pentru a determina dacă poate fi utilizată fără a genera conflicte suplimentare.

În cazul în care este identificat un interval valid, calendarul este actualizat automat, iar competiția își poate continua desfășurarea fără modificări semnificative asupra structurii existente. Dacă nu există o soluție care să respecte toate constrângerile definite, este necesară intervenția utilizatorului pentru stabilirea unei alternative.[11]

Modelul ilustrat evidențiază principiul de funcționare al mecanismelor de reprogramare implementate în cadrul aplicației și rolul acestora în menținerea unui calendar competițional valid pe întreaga durată a sezonului.

2.6 SIMULAREA COMPETIȚIILOR SPORTIVE

Simularea reprezintă o metodă utilizată frecvent pentru analizarea comportamentului unor sisteme complexe în condiții controlate. Prin intermediul simulării este posibilă reproducerea unor scenarii apropiate de realitate fără a fi necesară desfășurarea efectivă a proceselor analizate. Această abordare permite evaluarea performanței unui sistem, identificarea unor situații problematice și testarea diferitelor soluții înainte de aplicarea acestora într-un context real.[5]

În domeniul sportului, simularea competițiilor este utilizată pentru analizarea formatelor competiționale, evaluarea impactului anumitor reguli și studierea comportamentului diferitelor mecanisme de organizare. Astfel, simulările permite generarea unor volume mari de date care pot fi utilizate pentru testarea și validarea aplicațiilor software dedicate administrării competițiilor sportive.[16]

În cadrul lucrării, simularea nu reprezintă obiectivul principal al sistemului, ci un instrument utilizat pentru validarea mecanismelor de planificare și reconfigurare. Prin simularea unui sezon complet se poate observa modul în care calendarul competițional evoluează și modul în care sistemul reacționează atunci când apar situații ce necesită modificări ale planului inițial.

Pentru a fi cât mai apropiat de realitate, aplicația utilizează un model competițional inspirat din liga națională de baschet din România. Această simulare include gestionarea echipelor participante, a jucătorilor, rezultatele meciurilor și clasamentele generate pe parcursul unui sezon. Toate aceste elemente contribuie la crearea unui mediu realist în care pot fi evaluate funcționalitățile dezvoltate.[7][8]

Prin utilizarea simulării, acest sistem generează scenarii complete de desfășurare a unui sezon competițional, oferind posibilitatea analizării comportamentului mecanismelor de planificare și reprogramare la un nivel apropiat de cel din competițiile reale.



Figura 2.5 - Rolul simulării în cadrul sistemului

Figura 2.5 evidențiază rolul simulării în cadrul sistemului dezvoltat și modul în care aceasta contribuie la validarea mecanismelor de planificare și reconfigurare. Procesul pornește de la un set de date competiționale care include informații despre echipe, jucători, infrastructura disponibilă și regulile competiției.

Pe baza acestor informații este simulată desfășurarea unui sezon competițional complet, fiind generate meciurile și rezultatele asociate. Datele

obținute permit actualizarea clasamentelor și determinarea evoluției competiției pe parcursul diferitelor etape ale sezonului.[16]

Rezultatele produse în urma unei simulări sunt utilizate pentru evaluarea comportamentului sistemului în condiții apropiate de cele întâlnite în competițiile reale. [5]

Așadar, pot finalizate atât corectitudinea procesului de planificare, cât și capacitatea mecanismelor de reprogramare de a menține un calendar valid în prezența unor modificări sau evenimente neprevăzute.

Deci, această simulare nu reprezintă obiectivul principal al produsului, ci este doar un instrument de validare care oferă un mediu realist pentru testarea și evaluarea funcționalităților dezvoltate în cadrul lucrării.

3. ANALIZA ȘI PROIECTAREA SISTEMULUI

În urma analizei problemelor apărute în urma planificării și reconfigurării competițiilor sportive a fost proiectat un sistem software capabil să genereze și să administreze calendare competiționale într-un mod automatizat. Acest sistem urmărește integrarea mecanismelor de planificare, simulare și gestionare a evenimentelor într-o singură platformă software accesibilă prin intermediul unei interfețe web.

Procesul de proiectare are la bază identificarea funcționalităților necesare pentru administrarea unei competiții sportive complete și pentru simularea desfășurării unui sezon competițional. Astfel, au fost definite cerințele sistemului, principalele componente software și relațiile dintre acestea.

Acest capitol descrie cerințele funcționale și nefuncționale ale aplicației, actorii implicați în utilizarea sistemului și principalele decizii de proiectare adoptate în cadrul dezvoltării soluției.

3.1 CERINȚELE FUNCȚIONALE

Acestea descriu comportamentul sistemului și funcționalitățile pe care acesta trebuie să le ofere utilizatorului. În cadrul aplicației au fost identificate următoarele cerințe:

- ➔ generarea automată a unui sezon competițional complet
- ➔ generarea și administrarea calendarului competițional
- ➔ gestionarea echipelor participante și a loturilor de jucători
- ➔ simularea desfășurării meciurilor
- ➔ actualizarea clasamentelor pe baza rezultatelor generate
- ➔ organizarea competițiilor auxiliare (Supercupa României, Cupa României)
- ➔ gestionarea fazelor de play-off și play-out
- ➔ introducerea și administrarea evenimentelor ce afectează calendarul
- ➔ reprogramarea meciurilor afectate de apariția unor conflicte
- ➔ vizualizarea rezultatelor și a statisticilor generale
- ➔ exportarea datelor competiționale într-un format utilizabil

Aceste funcționalități reprezintă baza sistemului ce a fost dezvoltat și permit simularea și administrarea competiției complete într-un mod automatizat.



Figura 3.1 - Funcționalitățile principale ale sistemului

Figura 3.1 prezintă principalele funcționalități ale sistemului ce a fost proiectat și modul în care acestea contribuie la administrarea unei competiții sportive. În centrul aplicației se află componenta de management a competiției ce coordonează procesele de planificare, de simulare și de gestionare a datelor competiționale.

Sistemul permite generarea automată a calendarului competițional și administrarea echipelor participante, dar și a loturilor de jucători pentru fiecare echipă. Pe baza acestor informații, aplicația simulează desfășurarea unui sezon complet și generează rezultatele pentru actualizarea clasamentelor.

În plus, acest sistem software oferă mecanisme pentru gestionarea evenimentelor care influențează desfășurarea competiției și pentru reprogramarea meciurilor afectate de astfel de situații. În paralel, utilizatorul poate consulta statisticile și poate exporta datele generate în urma simulărilor efectuate.

Totalul funcționalităților prezentate contribuie la realizarea platformei integrate pentru planificare, simulare și administrare a competițiilor.

3.2 CERINȚELE NEFUNCȚIONALE

Pe lângă funcționalitățile oferite utilizatorului, sistemul trebuie să respecte o serie de cerințe nefuncționale care influențează calitatea și eficiența soluției prezentate. Aceste cerințe descriu caracteristici precum performanța, modularitatea, capacitatea de extindere sau ușurința în utilizare.

Prima cerință importantă este reprezentată de performanță. Generarea calendarului și procesul de simulare trebuie să fie realizate într-un timp decent, astfel utilizatorul poate interacționa eficient cu aplicația și poate obține rapid rezultatele așteptate.

Sistemul trebuie să asigure un nivel ridicat de fiabilitate, astfel datele competiționale să fie gestionate corect pe parcursul sezonului. Orice modificare efectuată asupra calendarului sau a rezultatelor trebuie să fie reflectată în toate componentele aplicației.

O altă cerință o reprezintă modularitatea. Arhitectura proiectului trebuie să permită extinderea sau modificarea funcționalităților existente fără a afecta comportamentul general al acestuia. Astfel, soluția are la bază o structură modulară care separă responsabilitățile între diferitele componente software.

Din punct de vedere al utilizabilității, aplicația oferă o interfață intuitivă și accesibilă. Utilizatorul trebuie să poată configura simulări, să consulte informațiile generate și să administreze competițiile fără a avea nevoie de cunoștințe tehnice exagerate.[14]

Portabilitatea reprezintă o caracteristică importantă deoarece utilizarea containerelor Docker permite distribuirea și rularea aplicației într-un mod uniform, indiferent de platforma hardware sau software utilizată.[5]

Prin respectarea acestor cerințe, sistemul poate oferi un grad ridicat de flexibilitate și poate constitui o bază solidă pentru dezvoltări ulterioare.

3.3 ACTORII SISTEMULUI

Actorii sistemului reprezintă entitățile ce interacționează direct cu aplicația și utilizează funcționalitățile puse la dispoziție de aceasta. Vis a vis de soluție, a fost identificat un actor principal, responsabil de configurarea și administrarea competiției simulate.

Acest actor al sistemului este utilizatorul aplicației. Acesta are acces la funcționalitățile necesare pentru generarea simulării, administrarea competiției și analizarea rezultatelor primite.

Prin intermediul interfeței web, utilizatorul inițiază simularea unui sezon competițional, poate consulta calendarul generat, poate urmări clasamentele și statisticile disponibile și poate introduce evenimente ce influențează desfășurarea competiției. Acesta mai poate analiza rezultatele obținute și poate exporta datele generate de aplicație.

Aceste interacțiuni dintre utilizator și sistem sunt posibile exclusiv prin intermediul interfeței web, aceasta comunică cu serviciile backend pentru executarea operațiilor solicitate și pentru afișarea informațiilor rezultate.

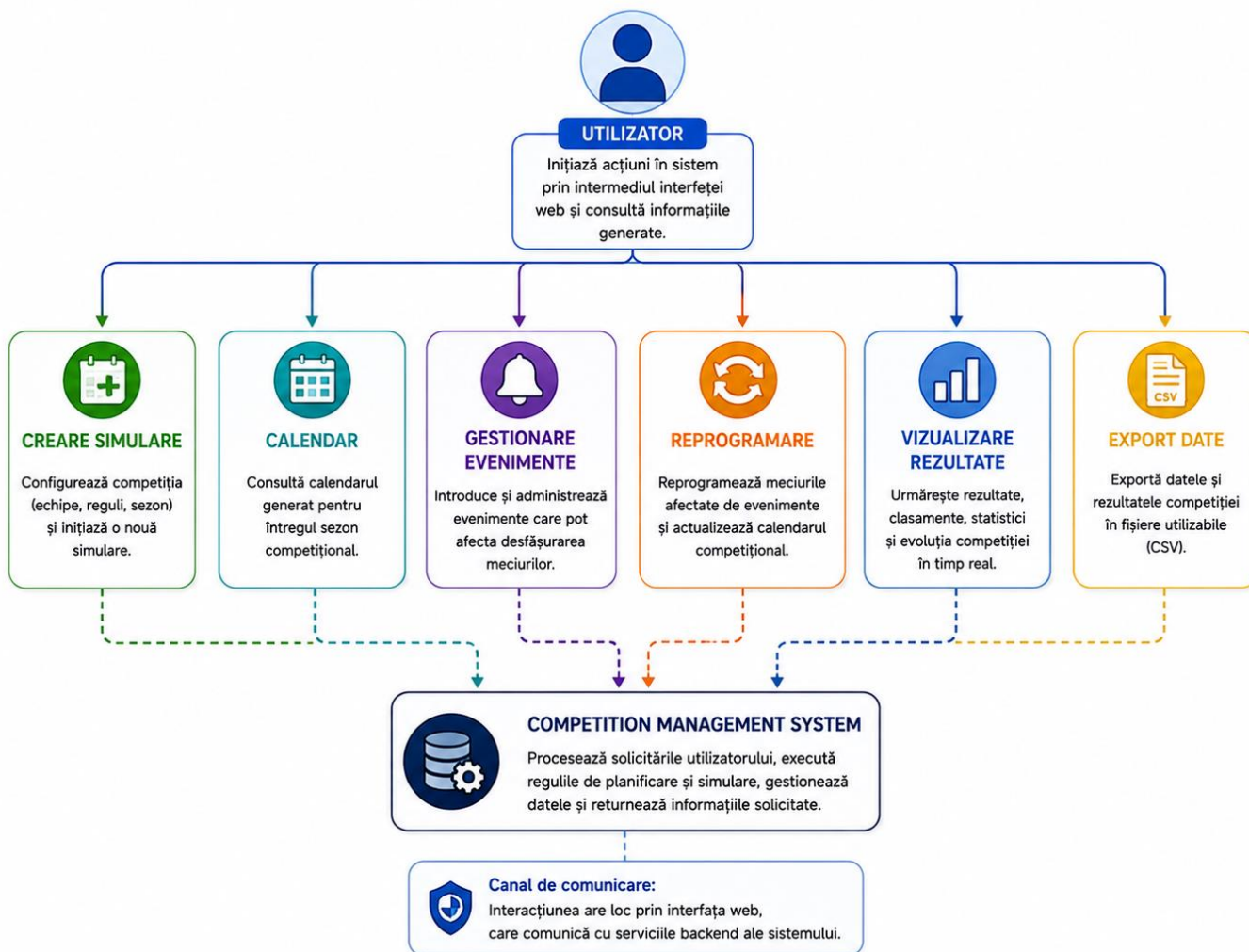


Figura 3.2 - Interacțiunea utilizatorului cu sistemul

Figura 3.2 prezintă principalele interacțiuni dintre utilizator și sistemul dezvoltat. Utilizatorul reprezintă actorul principal al aplicației și poate accesa funcționalitățile disponibile prin intermediul interfeței web.

Prin intermediul aplicației, utilizatorul poate configura și iniția simularea unui sezon competițional, poate consulta calendarul generat și poate administra evenimentele ce influențează desfășurarea competiției. Acest sistem oferă posibilitatea reprogramării meciurilor afectate de modificări ale calendarului, precum și acces la rezultate, clasamente, statistici generate pe parcursul simulării.

Toate solicitările efectuate de utilizator sunt procesate de sistemul central de administrare a competiției, care aplică regulile de planificare,

gestionează datele competiționale și furnizează informațiile necesare interfeței utilizator. Comunicarea dintre acesta și componente este realizată prin intermediul interfeței web și al serviciilor backend asociate.

Figura evidențiază rolul central al sistemului în coordonarea tuturor funcționalităților disponibile și modul în care acestea sunt accesate de către utilizator.

3.4 CAZURI DE UTILIZARE

Acestea descriu modul în care utilizatorul interacționează cu sistemul pentru a realiza diferite activități disponibile în aplicație. Acestea permit evidențierea principalelor funcționalități și oferă o imagine de ansamblu asupra modului de utilizare a sistemului.[14]

Principalul scenariu este reprezentat de crearea simulării competiționale. În cadrul procesului, utilizatorul configurează parametrii necesari și inițiază generarea unui sezon competițional complet. Sistemul generează calendarul competițional și pregătește datele necesare pentru simulare.

Un alt scenariu este consultarea calendarului. Utilizatorul poate analiza programarea meciurilor și poate urmări modul în care sunt distribuite etapele competiției pe parcursul sezonului.

Această aplicație permite și gestionarea evenimentelor ce influențează desfășurarea competiției. Utilizatorul poate introduce situații ce afectează calendarul, iar sistemul va analiza impactul și va identifica soluții de reprogramare dacă este necesar.

Pe parcursul simulării, utilizatorul poate consulta rezultatele generate, clasamentele actualizate și statisticile asociate echipelor și jucătorilor participanți. Aceste informații permit monitorizarea evoluției competiției și analiza performanțelor înregistrate.

Aplicația oferă posibilitatea exportării datelor generate în urma simulării efectuate. Astfel, utilizatorul poate salva și utiliza ulterior informațiile produse de sistem.

Cazurile de utilizare prezentate evidențiază principalele funcționalități ale aplicației și modul în care acestea contribuie la administrarea și simularea unei competiții sportive complete.

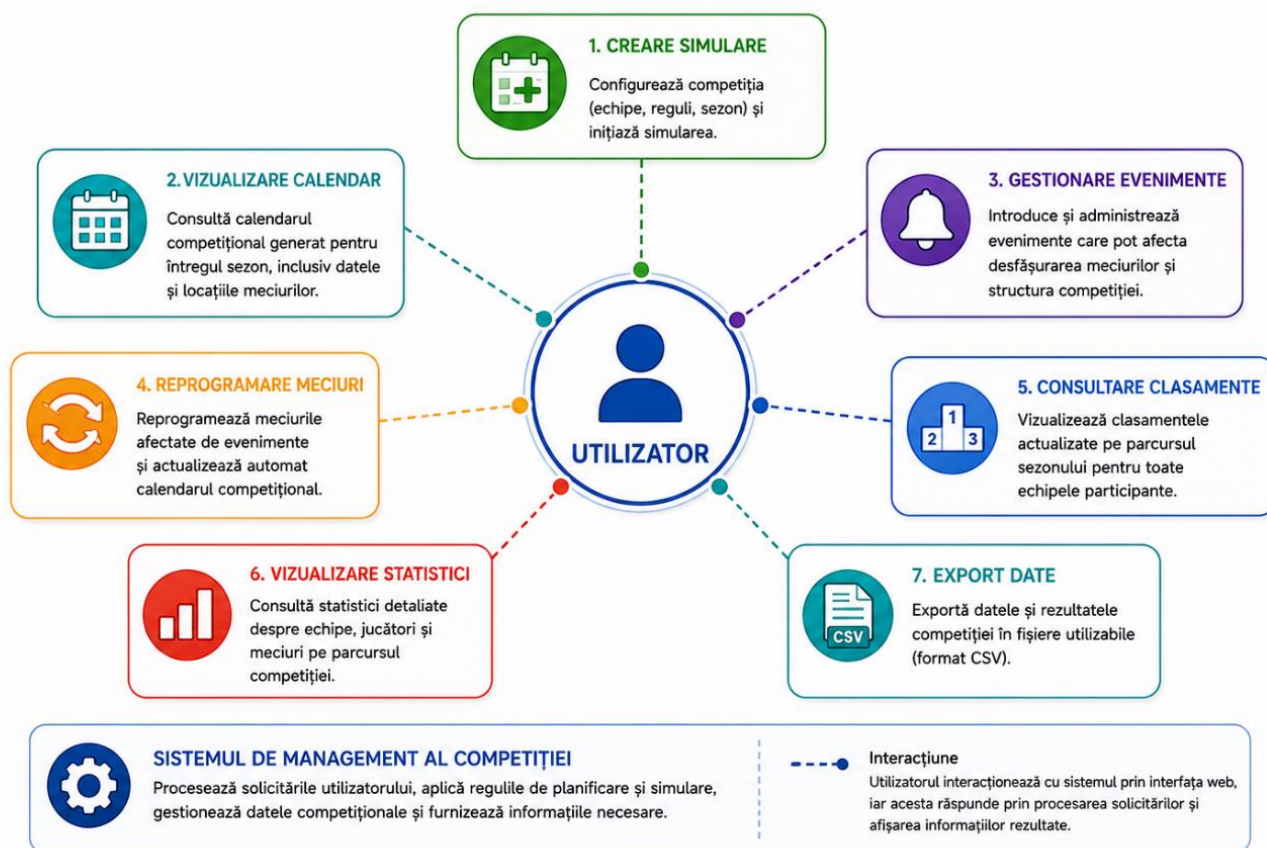


Figura 3.3 - Principalele cazuri de utilizare ale sistemului

Figura 3.3 prezintă principalele cazuri de utilizare ale sistemului și modul în care utilizatorul interacționează cu funcționalitățile disponibile în aplicație. Utilizatorul reprezintă actorul central al sistemului și poate accesa toate componentele necesare pentru administrarea și simularea unei competiții sportive.

Procesul începe prin configurarea și crearea unei simulări competiționale, în cadrul căreia sunt definite informațiile necesare desfășurării sezonului. Ulterior, utilizatorul poate consulta calendarul generat și poate monitoriza programarea meciurilor pe parcursul competiției.

Aplicația oferă posibilitatea introducerii și gestionării evenimentelor care pot influența desfășurarea normală a sezonului. În situațiile în care acestea generează conflicte de programare, sistemul poate efectua operații de reprogramare pentru identificarea unor intervale alternative valide.

Pe parcursul simulării, utilizatorul poate consulta clasamentele actualizate și statisticile generate pentru echipe și jucători. În plus, rezultatele

și informațiile produse de sistem pot fi exportate în formate utilizabile pentru analiză și arhivare.

Figura evidențiază rolul utilizatorului în cadrul aplicației și principalele funcționalități puse la dispoziție pentru gestionarea completă a unei competiții sportive simulate.

3.5 PROIECTAREA CONCEPTUALĂ A SISTEMULUI

Sistemul dezvoltat a fost proiectat utilizând o arhitectură modulară, în cadrul căreia funcționalitățile aplicației sunt împărțite în componente specializate. Această abordare ușurează dezvoltarea, extinderea și întreținerea aplicației, permițând separarea clară a responsabilităților între diferite module software.[14]

La nivel conceptual, sistemul este alcătuit din mai multe componente principale care colaborează pentru gestionarea unei competiții sportive. Interfața utilizator reprezintă punctul de interacțiune dintre utilizator și aplicație, oferind acces la funcționalități disponibile și la informațiile generate de sistem.

Componenta backend este responsabilă pentru procesarea solicitărilor primite de la utilizator și pentru coordonarea logicii aplicației. Aceasta gestionează operațiile de planificare, simulare, administrare a competiției și reprogramare a meciurilor.

Motorul de planificare are rolul de a genera calendarul competițional și de a verifica respectarea constrângerilor definite. În paralel, motorul de simulare gestionează desfășurarea sezonului competițional, generarea rezultatelor și actualizarea clasamentelor.

Persistența datelor este dată prin intermediul unei baze de date relaționale care stochează informațiile asociate competiției, echipelor participante, jucătorilor, meciurilor și rezultatelor generate pe parcursul simulării.[9]

Separarea funcționalităților în componente distincte contribuie la creșterea modularității aplicației și permite dezvoltarea independentă a diferitelor părți ale sistemului.

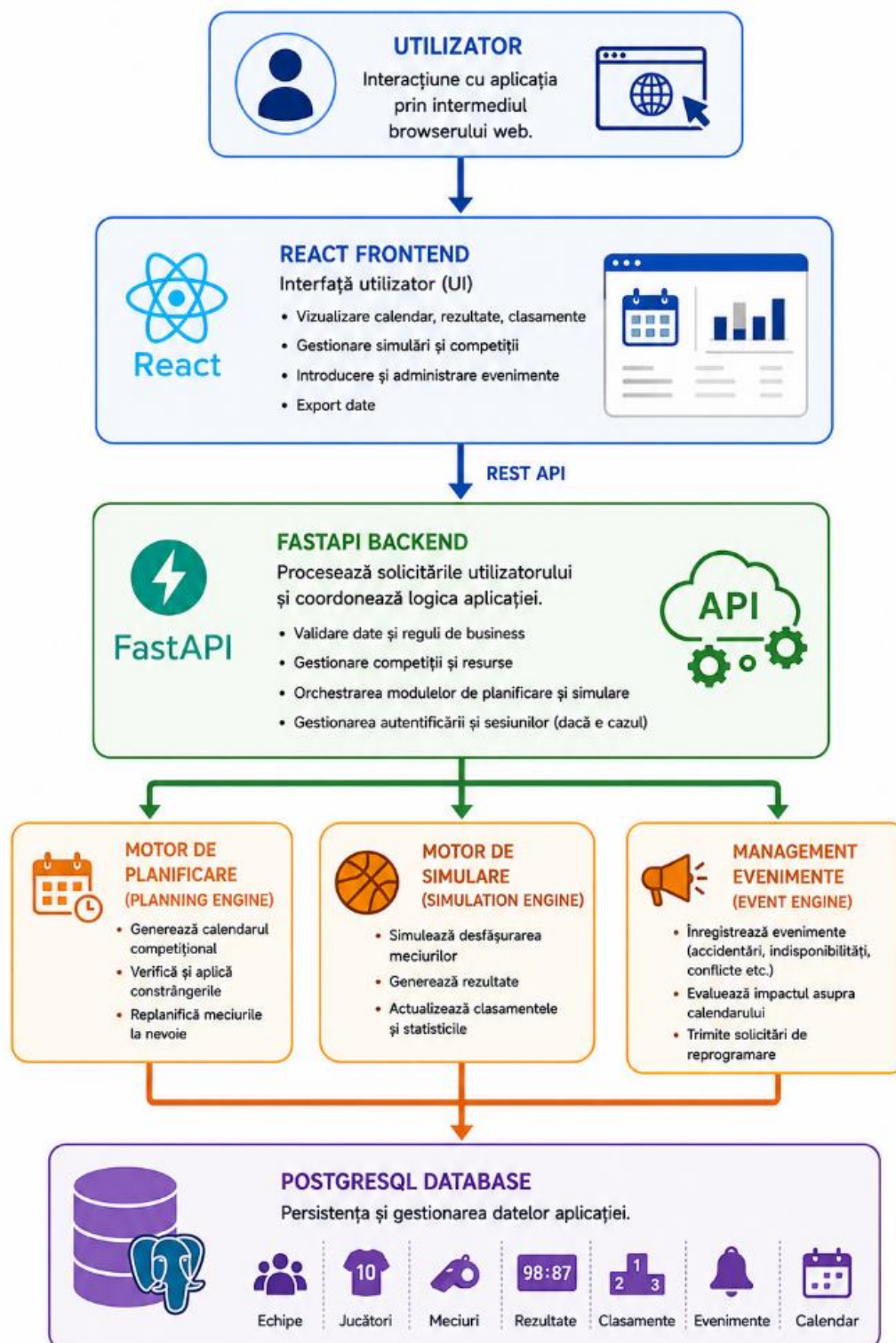


Figura 3.4 - Arhitectura conceptuală a sistemului

Figura 3.4 prezintă arhitectura conceptuală a sistemului dezvoltat și principalele componente care contribuie la funcționarea aplicației. Structura soluției urmează modelul client-server, utilizatorul având interacțiunea cu aplicația prin intermediul unei interfețe web implementate folosind tehnologia React.[9]

Interfața frontend este responsabilă pentru afișarea informațiilor și pentru colectarea acțiunilor efectuate de utilizator. Prin intermediul acesteia pot fi create simulări, consultate calendarele competiționale, administrate evenimentele și analizate rezultatele generate pe parcursul sezonului.

Comunicarea dintre frontend și backend este realizată cu ajutorul interfeței REST API. Componenta backend, implementată utilizând framework-ul FastAPI, procesează solicitările primite și coordonează funcționarea principalelor module ale aplicației.[16]

La nivelul backend-ului s-au găsit trei componente majore. Motorul de planificare este responsabil de generarea și actualizarea calendarului competițional, motorul de simulare gestionează desfășurarea sezonului și generarea rezultatelor, iar componenta de management a evenimentelor monitorizează situațiile ce pot afecta calendarul și inițiază procesele de reprogramare dacă e nevoie.

Persistența datelor este asigurată de baze de date PostgreSQL, ce stochează informațiile asociate echipelor, jucătorilor, meciurilor, rezultatelor, clasamentelor și evenimentelor competiționale. Organizarea sistemului în componente distincte contribuie la creșterea modularității aplicației și ușurează dezvoltarea și extinderea ulterioară a funcționalităților existente.[14]

3.6 COMPONENTELE PRINCIPALE ALE SISTEMULUI

Pentru a putea realiza toate aceste funcționalități, sistemul a fost împărțit în mai multe componente specializate, fiecare având responsabilități clare. Această separare contribuie la creșterea modularității aplicației și facilitează dezvoltarea și întreținerea acesteia.[10]

Componenta frontend reprezintă interfața prin care utilizatorul interacționează cu aplicația. Aceasta oferă acces la funcționalitățile de configurare a simulării, de vizualizare a calendarului, de consultare a clasamentelor și de analiză asupra statisticilor generate.[11]

Componenta backend coordonează logica principală a aplicației și procesează solicitările primite din partea utilizatorului. Aceasta asigură comunicarea dintre interfața web, modulele interne și baza de date.[17]

Motorul de planificare este responsabil de generarea calendarului competițional și de verificarea respectării constrângerilor stabilite. Acesta reprezintă una dintre componentele esențiale ale sistemului și contribuie la construirea unui calendar valid pentru desfășurarea competiției.

Motorul de simulare gestionează desfășurarea sezonului competițional și actualizarea informațiilor rezultate în urma disputării meciurilor. Acesta generează rezultatele partidelor și contribuie la actualizarea clasamentelor și statisticilor.

Componenta de gestionare a evenimentelor monitorizează situațiile care pot afecta desfășurarea competiției și transmite informațiile necesare către mecanismele de reprogramare. Prin intermediul acestei componente este posibilă simularea unor scenarii care impun modificarea calendarului competițional.

Persistența informațiilor este asigurată de baza de date care stochează toate datele asociate competiției și permite accesarea acestora de către celelalte componente ale sistemului.[14][17]

4. ARHITECTURA ȘI TEHNOLOGIILE UTILIZATE

Pentru dezvoltarea sistemului propus s-a folosit o arhitectură de tip client-server, care separă responsabilitățile asociate interfeței utilizator de cele asociate procesării datelor și logicii aplicației. Această abordare dă voie dezvoltării independente a componentelor software și facilitează extinderea ulterioară a sistemului.[14][9]

Soluția prezentată folosește tehnologii noi pentru dezvoltarea interfeței, dar și pentru implementarea serviciilor backend și gestionarea datelor. Componenta frontend a fost realizată utilizând biblioteca React, componenta backend utilizează framework-ul FastAPI, iar persistența informațiilor are loc cu ajutorul sistemului de gestiune a bazelor de date PostgreSQL.[17][11][14]

Pentru a simplifica procesul de distribuție și rulare a aplicației s-a folosit tehnologia Docker, ce permite containerizarea componentelor sistemului și executarea acestora într-un mediu controlat și reproductibil.[5]

Acum se vor prezenta principalele componente tehnologice utilizate în cadrul aplicației și rolul fiecăreia în funcționarea sistemului.

4.1 ARHITECTURA SOFTWARE

Sistemul software dezvoltat în această lucrare utilizează o arhitectură de tip client-server, realizată pentru a separa componentele responsabile de interacțiunea cu utilizatorul de cele care implementează logica aplicației și gestionează datele competiționale. Astfel, este facilitată dezvoltarea independentă a componentelor sistemului și contribuie la creșterea modularității și a flexibilității soluției propuse.[14]

Arhitectura este organizată pe mai multe straturi, fiecare având responsabilități bine definite. La nivelul superior se află componenta frontend, adăugată cu ajutorul bibliotecii React, ce oferă utilizatorului acces la funcționalitățile sistemului prin intermediul interfeței web. Această componentă are rolul de a afișa informațiile și de a transmite solicitările către serviciile backend.[9][11]

Procesarea cererilor și coordonarea logicii aplicației sunt realizate de componenta backend, implementată utilizând framework-ul FastAPI. Backend-ul expune un set de endpoint-uri REST prin intermediul cărora sunt accesate funcționalitățile sistemului și sunt schimbate date între interfață și componentele interne ale aplicației.[16][17]

Pentru organizarea logicii aplicației s-a folosit o structură modulară. Componenta API gestionează comunicarea cu frontend-ul și expune funcțiile disponibile. Modulele dedicate competițiilor și evenimentelor implementează operațiile specific administrării competiției și gestionării situațiilor ce pot influența desfășurarea sezonului. Componenta use cases coordonează scenariile principale de utilizare și procesele de business implementate în aplicație.[14]

Regulile de domeniu și entitățile fundamentale ale sistemului sunt definite în cadrul stratului domain. Acesta conține modelele asociate competițiilor, echipelor, jucătorilor, meciurilor, rezultatelor, clasamentelor și mecanismelor de planificare. Separarea acestor entități de celelalte componente permite menținerea unei structuri clare și ușurează extinderea ulterioară a sistemului.[10]

Accesul la resursele externe și operațiile tehnice sunt gestionate prin intermediul stratului infrastructură. Acesta adaugă mecanismele de care avem nevoie pentru a comunica cu baza de date și pentru a realiza operațiile auxiliare necesare funcționării aplicației.

Persistența datelor este asigurată de sistemul de gestiune a bazelor de date PostgreSQL, utilizat pentru a stoca informațiile asociate competițiilor, echipelor, jucătorilor, meciurilor, rezultatelor și evenimentelor. Această componentă reprezintă nivelul inferior al arhitecturii și oferă suport pentru toate operațiile de citire și actualizare a datelor.[14]

Arhitectura software a produsului este prezentată în figura de mai jos și evidențiază relațiile dintre principalele componente ale sistemului și fluxul informațiilor dintre ele.

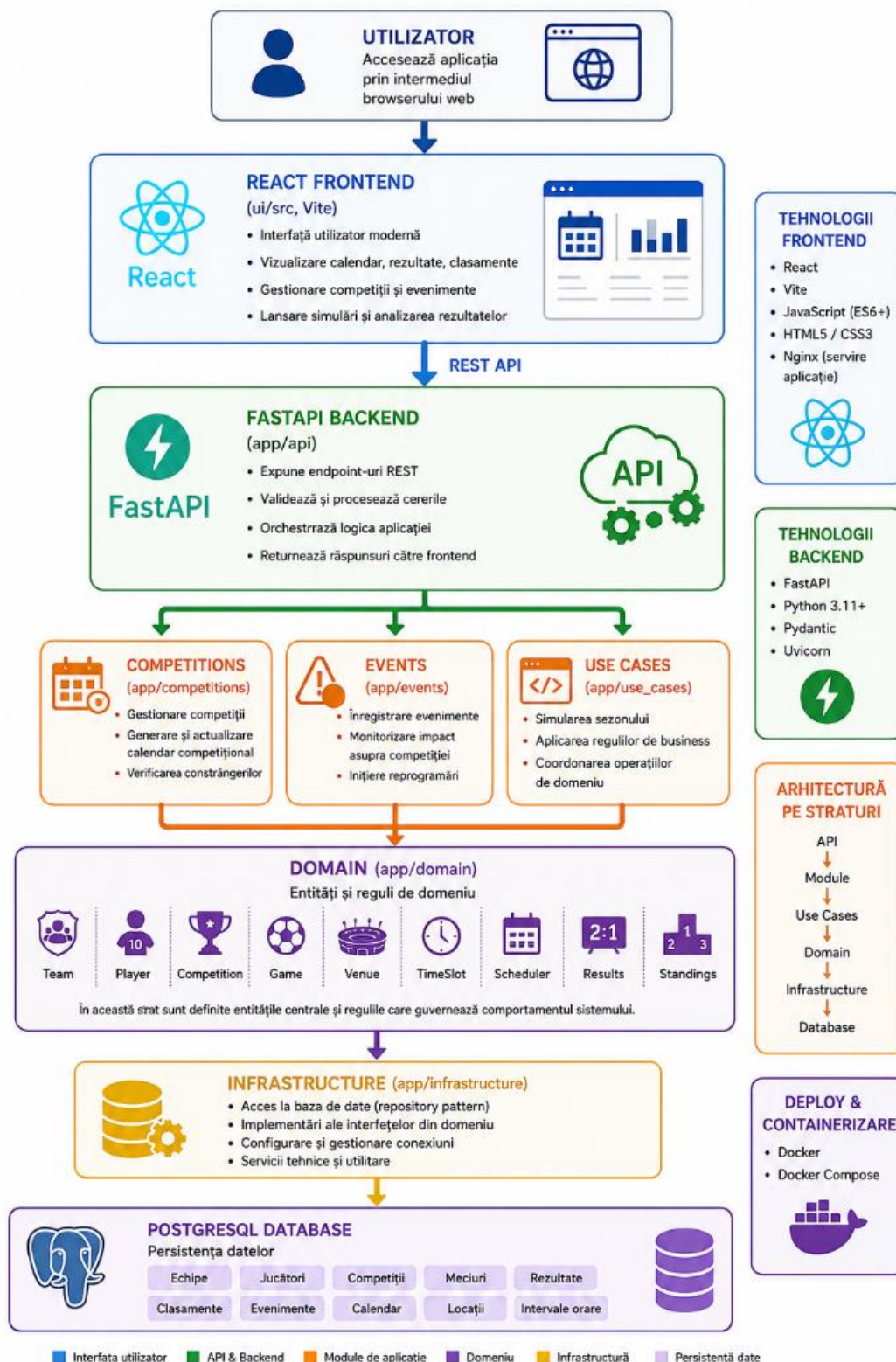


Figura 4.1 - Arhitectura software a aplicației

Figura prezintă arhitectura software a sistemului dezvoltat și principalele componente care contribuie la funcționarea aplicației. Soluția utilizează o arhitectură client-server, în cadrul căreia interfața utilizator, logica aplicației și persistarea datelor sunt separate în straturi distincte.

La nivel superior se află utilizatorul, care interacționează cu sistemul prin intermediul unei aplicații web dezvoltate folosind React. Componenta frontend este responsabilă pentru afișarea informațiilor și pentru transmiterea solicitărilor către serviciile backend prin intermediul unei interfețe REST.[16][11]

Componenta backend, implementată cu ajutorul framework-ului FastAPI, coordonează procesarea cererilor și comunicarea dintre diferitele module ale aplicației. În cadrul acesteia sunt integrate componente dedicate administrării competițiilor, gestionării evenimentelor și executării principalelor scenarii de utilizare ale sistemului.[17]

Logica de domeniu este organizată într-un strat separat care conține entitățile fundamentale ale proiectului, precum echipe, jucători, fiecare fază a competiției, rezultatele, clasamentele. Acest strat implementează regulile de business și reprezintă nucleul funcțional al sistemului.

Operațiile tehnice și accesul la resursele externe sunt realizate prin intermediul stratului de infrastructură, care asigură comunicarea cu baza de date și implementarea mecanismelor necesare funcționării aplicației.

La baza arhitecturii se află sistemul PostgreSQL, folosit pentru stocarea persistentă a informațiilor asociate competiției. Acesta permite gestionarea eficientă a datelor și oferă suport pentru toate operațiile de citire și actualizare efectuate de celelalte componente ale sistemului.[14]

Arhitectura prezentată evidențiază separarea responsabilităților între componentele aplicației și contribuie la creșterea modularității, mentenabilității și posibilităților de extindere ale sistemului software.

4.2 COMPONENTA FRONTEND

Aceasta reprezintă partea vizibilă a aplicației și asigură interacțiunea dintre utilizator și funcționalitățile sistemului. Aceasta a fost dezvoltată utilizând biblioteca React, una dintre cele mai populare soluții pentru dezvoltarea aplicațiilor web bazate pe componente reutilizabile.[11]

Interfața oferă acces la principalele funcționalități ale sistemului, printre care se numără configurarea simulărilor, consultarea calendarului competițional, gestionarea evenimentelor, vizualizarea rezultatelor și analiza statisticilor generate pe parcursul sezonului. Prin intermediul componentei, utilizatorul poate accesa toate informațiile relevante fără a interacționa direct cu serviciile backend sau cu baza de date.

Dezvoltarea aplicației frontend a fost realizată folosind Vite, un instrument pentru construirea și rularea aplicațiilor React. Acesta contribuie la reducerea timpului de compilare și oferă un mediu eficient pentru dezvoltarea și testarea interfeței.[11]

Structura proiectului frontend este organizată în jurul directorului src, care conține componentele și logica necesare funcționării aplicației. Resursele statice sunt gestionate prin directorul public, iar configurările necesare procesului de dezvoltare și distribuție sunt definite prin fișierele specifice mediului React și Vite.

Comunicarea dintre frontend și backend este realizată prin intermediul serviciilor REST expuse de aplicația FastAPI. Solicitățile trimise de utilizator sunt convertite în cereri HTTP către backend, iar răspunsurile primite sunt utilizate pentru actualizarea interfeței și afișarea informațiilor relevante.[16][17]

Prin utilizarea tehnologiilor React și Vite, aplicația beneficiază de o interfață modernă, flexibilă și ușor de extins, capabilă să ofere o experiență de utilizare eficientă pentru administrarea și simularea competițiilor sportive.[11]

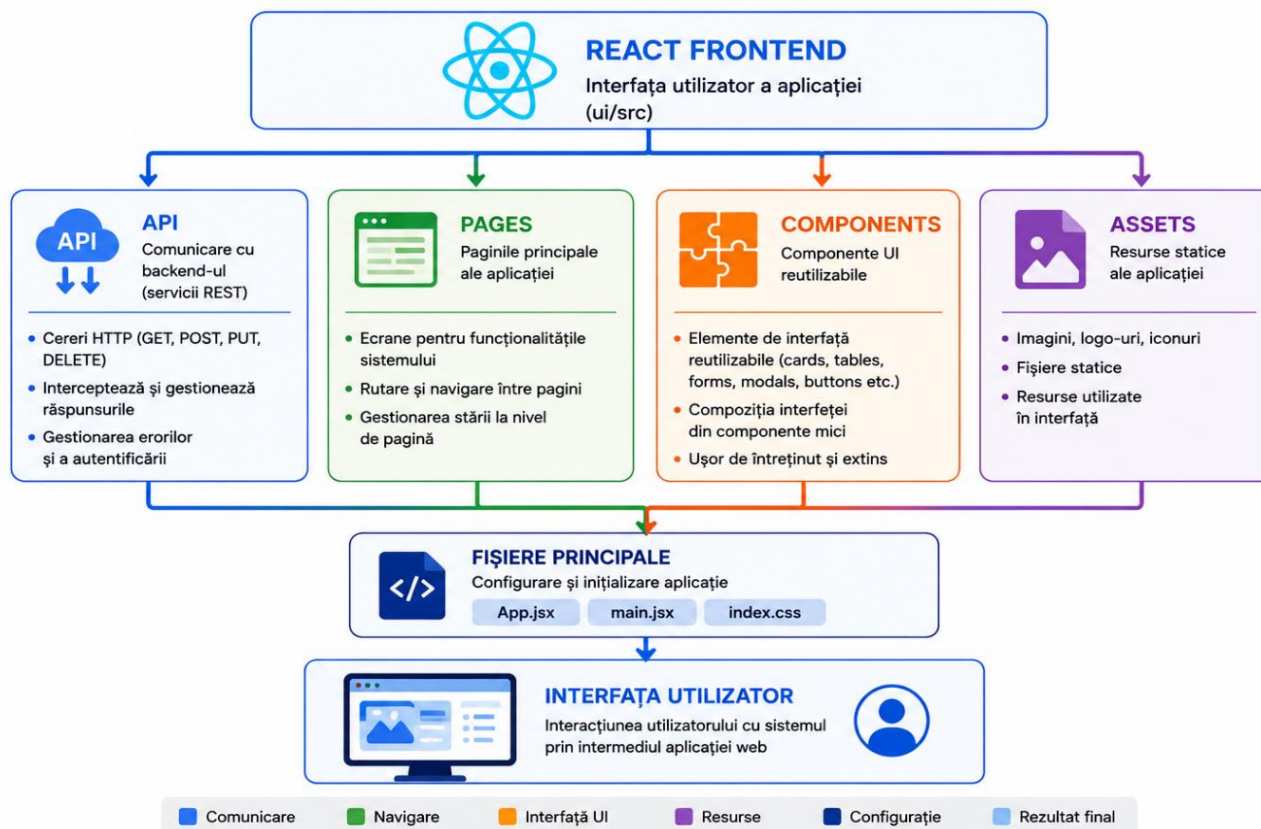


Figura 4.2 - Structura componente frontend

Figura 4.2 prezintă structura principală a componente frontend implementate utilizând biblioteca React. Organizarea aplicației urmărește separarea responsabilităților între diferitele module ale interfeței și facilitează dezvoltarea și extinderea ulterioară a funcționalităților.[11]

Directorul src reprezintă nucleul aplicației frontend și conține componentele necesare funcționării interfeței. Modulul api este responsabil de comunicarea cu serviciile backend și de transmitere a solicitărilor către server. Directorul assets conține resursele statice utilizate în aplicație, precum imagini și alte elemente grafice.

Funcționalitățile vizibile utilizatorului sunt organizate în cadrul directorului pages, care conține paginile principale ale aplicației. Componentele reutilizabile sunt grupate în directorul components, permițând reutilizarea elementelor de interfață în mai multe zone ale aplicației.

Fișierele App.jsx, main.jsx și index.css sunt utilizate pentru configurarea și inițializarea aplicației React și pentru definirea stilurilor generale ale

interfeței. Împreună, aceste componente contribuie la realizarea unei interfețe modern și modulare pentru administrarea competițiilor sportive.

4.3 COMPONENTA BACKEND

Această componentă reprezintă nucleul funcțional al aplicației și este responsabilă pentru implementarea logicii de business, procesarea solicitărilor primite din partea utilizatorului și coordonarea operațiilor asociate competiției sportive. Astfel, a fost dezvoltată utilizând framework-ul FastAPI, o soluție pentru dezvoltarea serviciilor web bazate pe limbajul Python.[17]

Backend-ul expune un set de endpoint-uri REST care permit comunicarea cu interfața și accesarea funcționalităților disponibile în aplicație. Prin intermediul serviciilor sunt gestionate operații de simulare, de administrare a competițiilor, de procesare a evenimentelor și de actualizare a datelor competiționale.[16]

Structura acestuia este organizată în mai multe module specializate. Componenta api gestionează expunerea serviciilor web și comunicarea cu frontend-ul. Modulul competitions implementează operațiile asociate organizării și desfășurării competițiilor sportive, în timp ce modulul events gestionează evenimentele care pot afecta calendarul competițional.

Scenariile principale de utilizare sunt coordonate prin intermediul componentei use_cases, care implementează procesele de business utilizate în cadrul aplicației. În versiunea actuală a sistemului, această componentă include mecanismele necesare simulării unui sezon competițional complet.[14]

Entitățile fundamentale ale sistemului sunt definite în cadrul stratului domain. Acesta conține modelele asociate echipelor, jucătorilor, competițiilor, meciurilor, rezultatelor, clasamentelor și mecanismelor de planificare. Separarea logicii de domeniu contribuie la creșterea modularității și facilitează reutilizarea componentelor în diferite scenarii.

Operațiile tehnice și accesul la resursele externe sunt implementate prin intermediul stratului infrastructură. Acesta oferă suport pentru comunicarea cu baza de date și pentru implementarea serviciilor necesare funcționării aplicației.

Prin utilizarea unei structuri modulare și a framework-ului FastAPI, componenta backend oferă o bază solidă pentru implementarea

funcționalităților de planificare, simulare și reconfigurare a competițiilor sportive.[17]

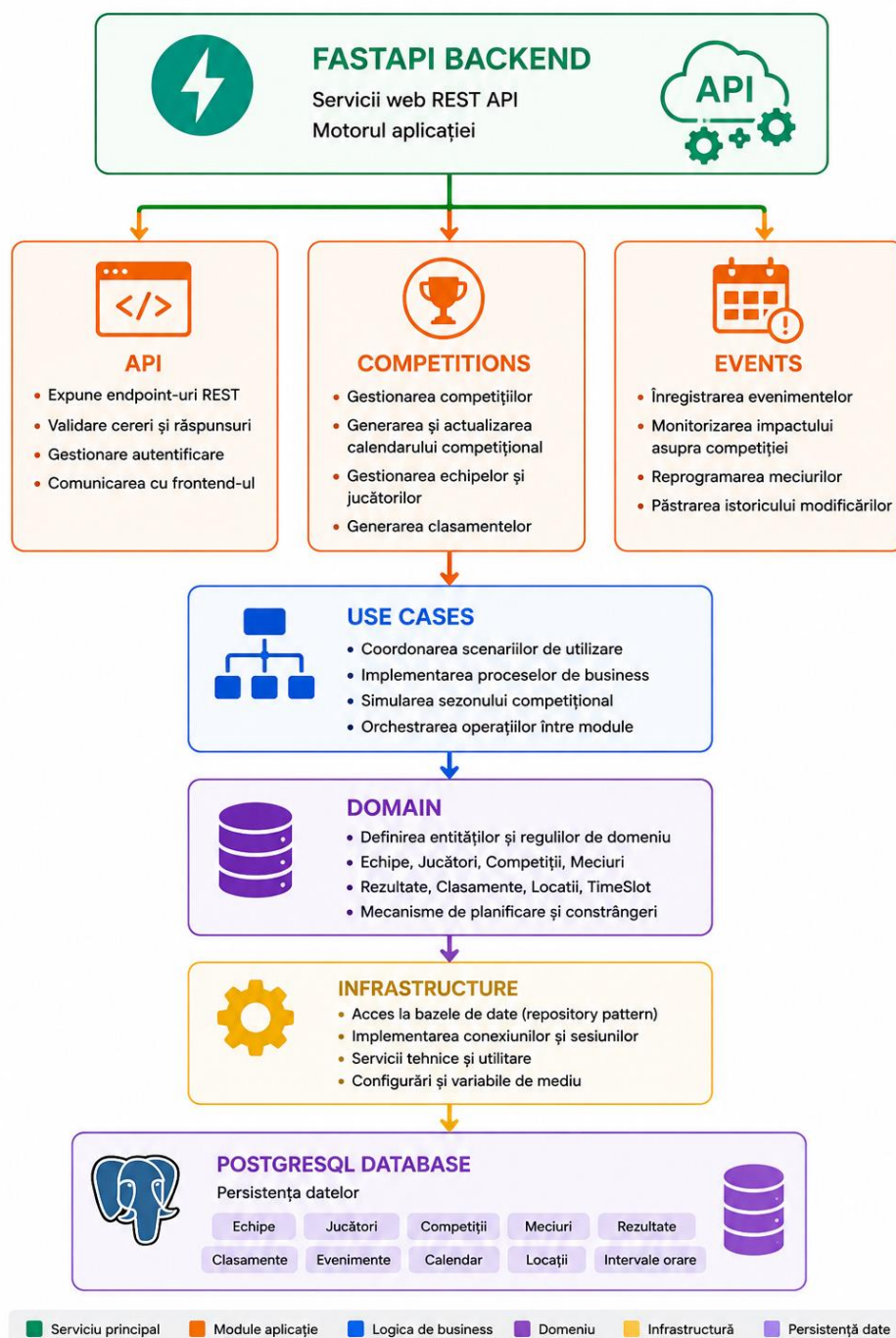


Figura 4.3 - Structura componente backend

Figura 4.3 ilustrează organizarea backend-ului în mai multe straturi funcționale. Componenta API gestionează comunicarea cu frontend-ul, modulele Competitions și Events implementează funcționalitățile specifice aplicației, iar stratul Domain conține entitățile și regulile de business. Persistența datelor este asigurată prin intermediul PostgreSQL.[17][14]

4.4 MODELUL DE DOMENIU

Acesta reprezintă baza conceptuală a aplicației și descrie entitățile principale utilizate pentru reprezentarea unei competiții sportive. Acesta definește obiectele fundamentale ale sistemului și relațiile existente între acestea, oferind suport pentru implementarea funcționalităților de planificare, simulare și administrare a competițiilor.[10]

În cadrul aplicației au fost definite mai multe entități care descriu elementele specifice unei competiții sportive. Echipele participante sunt reprezentate prin intermediul entității Team, care conține informațiile necesare identificării și administrării fiecărei participante în competiție. Fiecare echipă este asociată unui lot de jucători, modelat prin intermediul entității Player.

Desfășurarea competiției este reprezentată prin entitatea Competition, care gestionează structura competițională și relațiile dintre echipele participante. Meciurile disputate în cadrul competiției sunt modelate prin entitatea Game, care stochează informații despre participante, rezultate și programarea partidelor.

Planificarea calendarului competițional este realizată prin intermediul entităților TimeSlot și Venue, utilizate pentru reprezentarea intervalelor disponibile și a locațiilor în care pot fi programate meciurile. Aceste entități permit implementarea mecanismelor de planificare și verificare a constrângerilor asociate desfășurării competiției.

Pentru administrarea rezultatelor și a clasamentelor sunt utilizate entitățile Results și Standings, care permit monitorizarea evoluției competiției și actualizarea informațiilor generate pe parcursul sezonului.

Modelul de domeniu constituie nucleul funcțional al aplicației și oferă suport pentru toate procesele implementate în cadrul sistemului.[9]

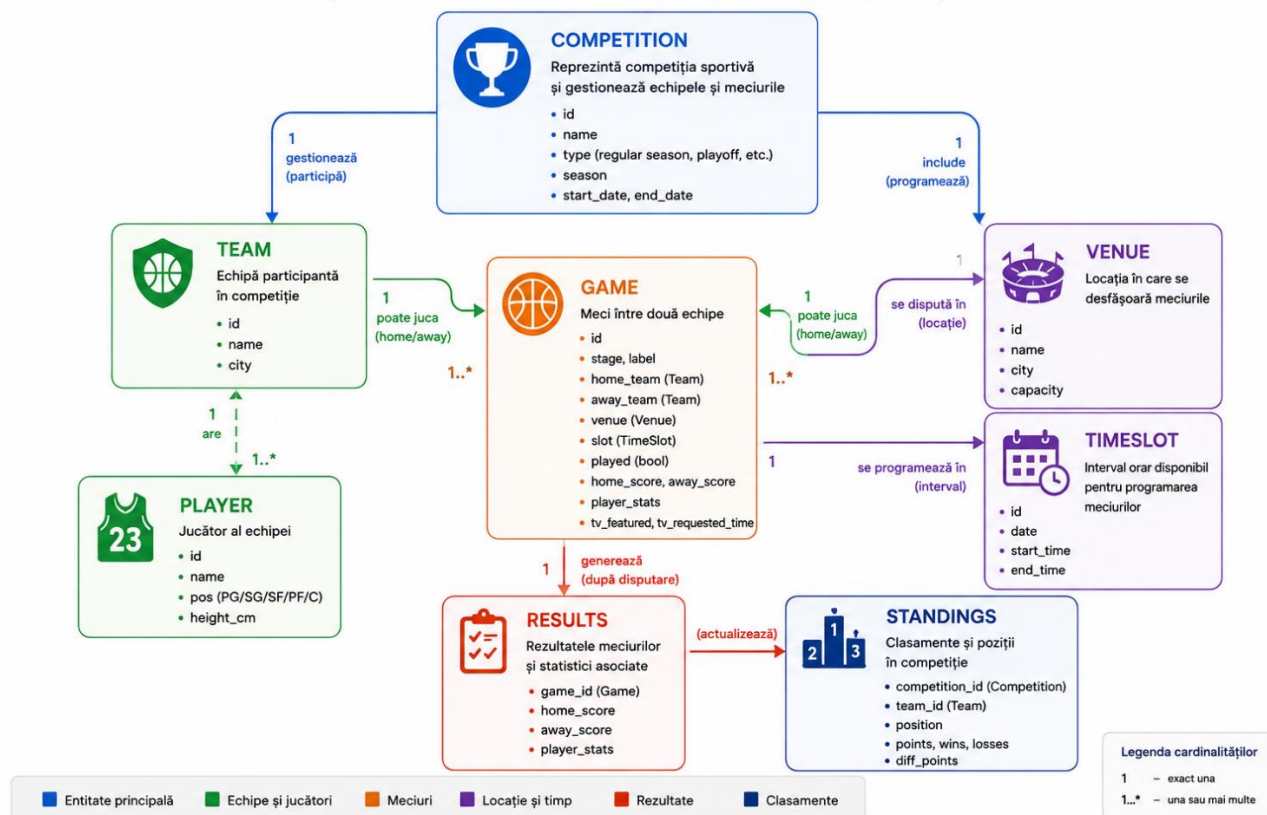


Figura 4.4 - Modelul de domeniu al aplicației

Figura 4.4 prezintă principalele entități ale modelului de domeniu și relațiile dintre acestea. Competiția reprezintă elementul central al sistemului și gestionează echipele participante și meciurile programate. Fiecare echipă este asociată unui set de jucători, iar fiecare meci este caracterizat prin participanți, locație și interval de desfășurare. Rezultatele obținute în urma disputării meciurilor sunt utilizate pentru actualizarea clasamentelor și monitorizarea evoluției competiției. Modelul de domeniu constituie baza funcțională a aplicației și este utilizat de toate componentele sistemului.

4.5 PERSISTENȚA DATELOR

Reprezintă mecanismul prin care informațiile generate și utilizate de aplicație sunt stocate și păstrate pe termen lung. În cadrul sistemului dezvoltat, această funcționalitate este realizată utilizând sistemul de gestiune

a bazelor de date PostgreSQL, ales datorită fiabilității, performanței și suportului extins pentru aplicații moderne.[14]

Baza de date este utilizată pentru stocarea informațiilor asociate competițiilor sportive, echipelor participante, jucătorilor, meciurilor, rezultatelor și clasamentelor generate pe parcursul simulărilor. Prin intermediul acesteia, sistemul poate păstra și reutiliza datele necesare administrării competiției fără a fi necesară regenerarea acestora la fiecare execuție a aplicației.

Accesul la baza de date este realizat prin intermediul stratului de infrastructură al aplicației, care izolează logica de business de detaliile tehnice asociate operațiilor de stocare și recuperare a informațiilor. Această abordare contribuie la creșterea modularității și facilitează eventualele modificări ale mecanismelor de persistență în viitor.[17]

Organizarea datelor într-o bază de date relațională permite menținerea integrității informațiilor și realizarea eficientă a operațiilor de căutare, actualizare și analiză a datelor generate de sistem. În plus, utilizarea PostgreSQL oferă suport pentru gestionarea unui volum ridicat de informații și pentru extinderea ulterioară a aplicației.[14]

Prin intermediul mecanismelor de persistență implementate, acest sistem poate administra în mod eficient toate informațiile necesare desfășurării și simulării competițiilor sportive.

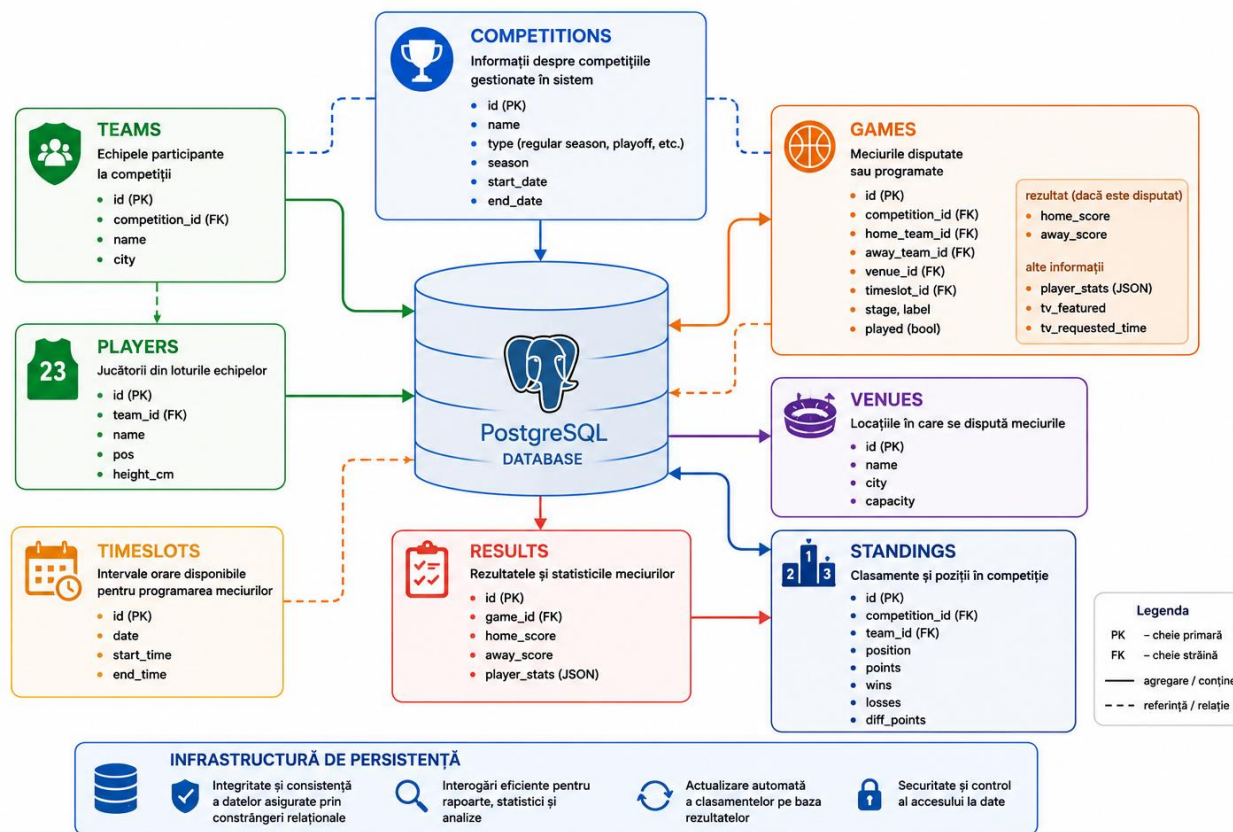


Figura 4.5 - Structura datelor gestionate de aplicație

Figura 4.5 prezintă principalele categorii de date gestionate de sistem și modul în care acestea sunt stocate în baza de date PostgreSQL. Informațiile asociate competițiilor, echipelor, jucătorilor, meciurilor, locațiilor și intervalelor de programare sunt organizate în entități distincte, care permit administrarea eficientă a sezonului competițional.[14]

Rezultatele generate în urma desfășurării meciurilor sunt utilizate pentru actualizarea clasamentelor și pentru calcularea statisticilor asociate competiției. Structura relațională a datelor contribuie la menținerea consistenței informațiilor și facilitează realizarea operațiilor de căutare, actualizare și analiză necesare funcționării aplicației.

4.6 CONTAINERIZAREA APLICAȚIEI

Pentru simplificarea procesului de instalare, configurare și distribuție a sistemului a fost utilizată tehnologia Docker. Containerizarea permite

împachetarea aplicației împreună cu toate dependențele necesare într-un mediu izolat, care poate fi executat în mod identic pe diferite sisteme de operare și configurații hardware.[5]

În cadrul proiectului au fost definite containere separate pentru componenta frontend și cea backend. O astfel de abordare permite dezvoltarea și administrarea independentă a fiecărei componente, ușurând actualizarea și extinderea aplicației fără a afecta funcționarea întregului sistem.

Configurarea și orchestrarea serviciilor sunt realizate prin intermediul fișierului `docker-compose.yml`, care descrie componentele necesare funcționării aplicației și relațiile dintre acestea. Prin utilizarea acestui mecanism, întregul sistem poate fi pornit și administrat printr-o singură comandă.[]

Containerizarea contribuie la creșterea portabilității aplicației și elimină problemele asociate diferențelor dintre mediile de dezvoltare și cele de producție. În plus, această abordare facilitează testarea, distribuirea și întreținerea sistemului pe termen lung.

Prin utilizarea tehnologiilor Docker și Docker Compose, soluția dezvoltată beneficiază de un proces simplificat de implementare și de un nivel ridicat de flexibilitate în ceea ce privește distribuția și rularea aplicației.[5]

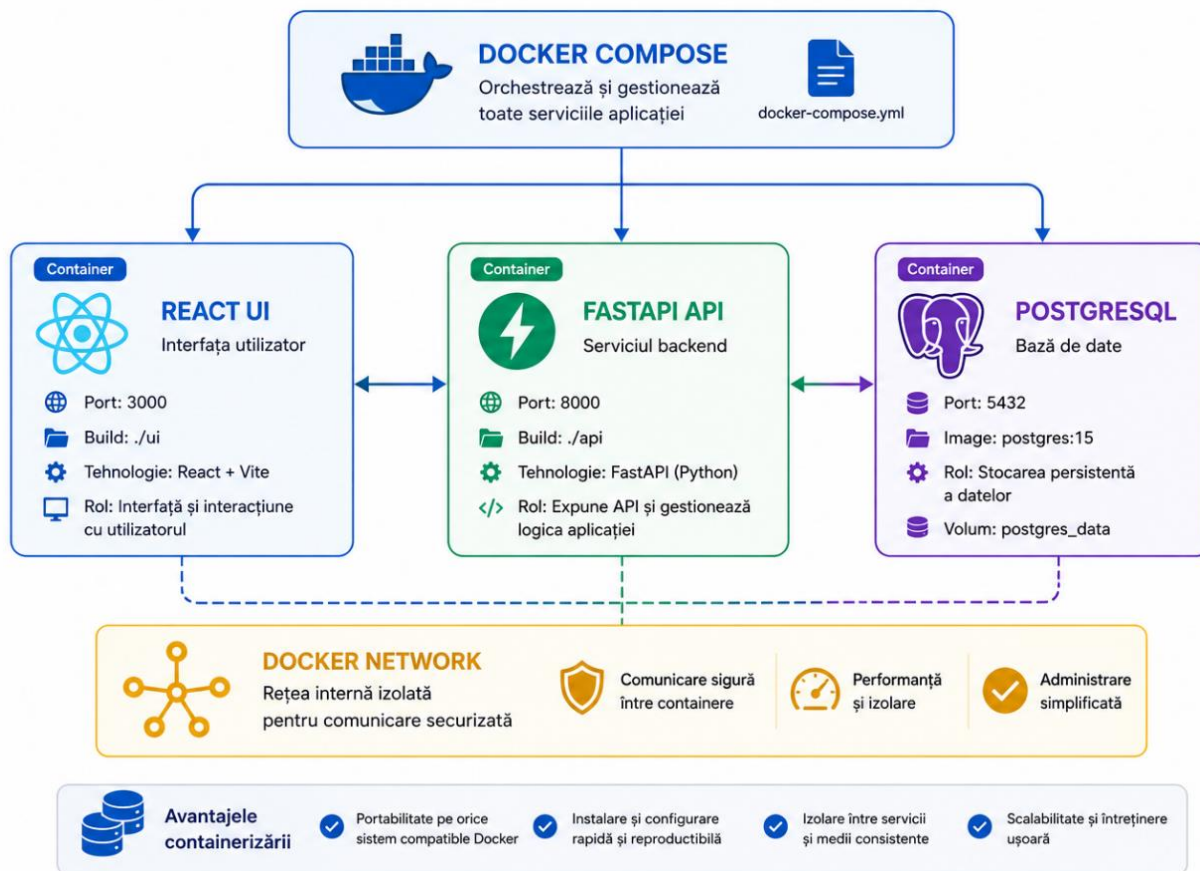


Figura 4.6 - Arhitectura containerizată a aplicației

Figura 4.6 prezintă modul în care aplicația este distribuită și executată utilizând tehnologia Docker. Sistemul este alcătuit din trei containere principale: componenta frontend implementată cu React, componenta backend dezvoltată folosind FastAPI și baza de date PostgreSQL utilizată pentru stocarea informațiilor aplicației.[5]

Orchestrarea și gestionarea serviciilor sunt realizate prin intermediul Docker Compose, care permite pornirea și administrarea tuturor componentelor folosind o configurație unitară. Comunicarea dintre containere este realizată prin intermediul unei rețele Docker dedicate, care asigură schimbul de date între servicii într-un mod controlat și securizat.

Utilizarea containerizării contribuie la creșterea portabilității aplicației și simplifică procesul de instalare, configurare și distribuție. În plus, această abordare permite rularea sistemului în condiții identice pe diferite platforme și facilitează dezvoltarea și întreținerea ulterioară a aplicației.

4.6.1 MODURI DE DISTRIBUIRE ȘI ACCES

Pe lângă rularea locală prin intermediul infrastructurii containerizate, aplicația League Pilot este disponibilă și într-o variantă online, găzduită în cloud prin intermediul platformei Railway.

Varianta locală este utilizată în principiu în etapa de dezvoltare și testare, oferind control complet asupra componentelor sistemului și facilitând procesul de depanare și validare.

Varianta online permite accesarea directă a aplicației prin intermediul browser-ului, fără a necesita instalarea locală a componentelor software. În cadrul implementării online, fiecare componentă principală a sistemului este distribuită ca serviciu separat:

- ➔ Componenta frontend React, responsabilă pentru interfața utilizator
- ➔ Componenta backend FastAPI, responsabilă pentru logica aplicației și expunerea serviciilor REST
- ➔ Baza de date PostgreSQL, responsabilă pentru persistența datelor generate în urma simulărilor

Această arhitectură asigură păstrarea acelorași funcționalități atât în mediul local, cât și în mediul online, inclusiv generarea simulărilor, vizualizarea rulărilor, descărcarea fișierelor exportate și administrarea acestora.

Frontend:

<https://frontend-production-dce1.up.railway.app>

Backend API:

<https://league-pilot-production.up.railway.app/docs>

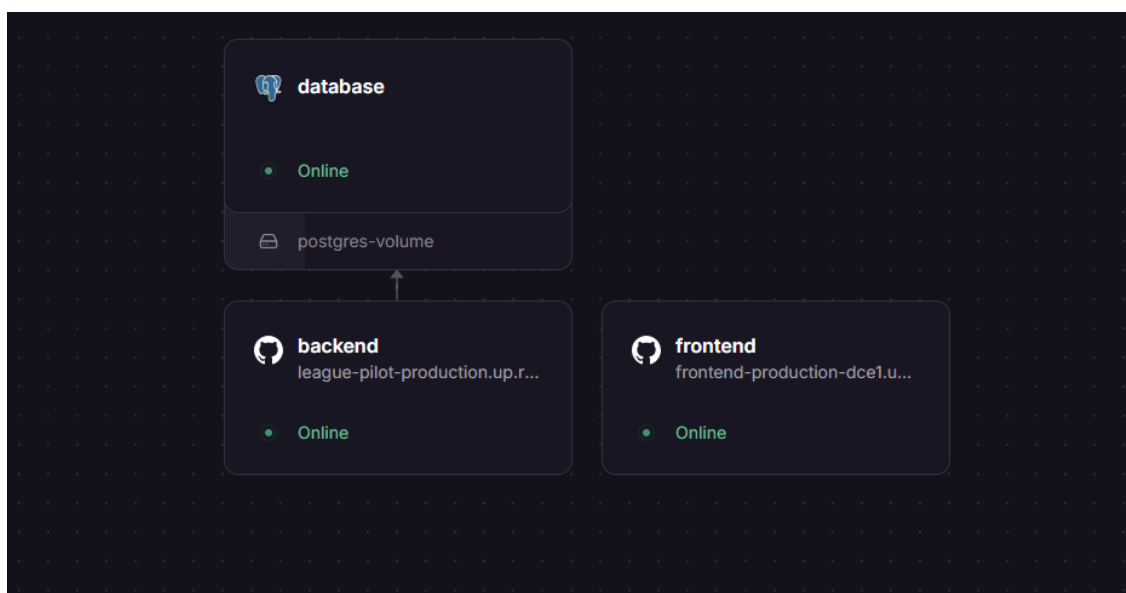


Figura 4.7 - Implementarea și orchestrarea componentelor aplicației în mediul cloud utilizând Railway (frontend, backend și baza de date PostgreSQL)

5. IMPLEMENTAREA SISTEMULUI

5.1 STRUCTURA PROIECTULUI

Implementarea sistemului a fost realizată utilizând o structură modulară care separă funcționalitățile aplicației în componente distincte. Această organizare facilitează dezvoltarea, întreținerea și extinderea ulterioară a sistemului, permițând gestionarea eficientă a logicii de business și a componentelor tehnice.

Proiectul este împărțit în două componente principale: backend și frontend. Componenta backend conține logica aplicației și mecanismele necesare administrării competițiilor sportive, în timp componenta frontend oferă interfața prin care utilizatorul interacționează cu sistemul.

În cadrul backend-ului sunt definite module dedicate gestionării competițiilor, administrării evenimentelor, implementării scenariilor de utilizare și modelării entităților de domeniu. Separarea acestora permite organizarea clară a responsabilităților și reducerea dependențelor dintre componente.

Componenta frontend este organizată în jurul paginilor și componentelor reutilizabile utilizate pentru construirea interfeței grafice. Comunicarea dintre frontend și backend este realizată prin intermediul serviciilor REST expuse de aplicația FastAPI.[16][17]

Pentru distribuirea aplicației au fost utilizate containere Docker, iar configurarea serviciilor este realizată prin intermediul fișierului Docker Compose. Această abordare permite rularea întregului sistem într-un mediu controlat și reproductibil.[5]

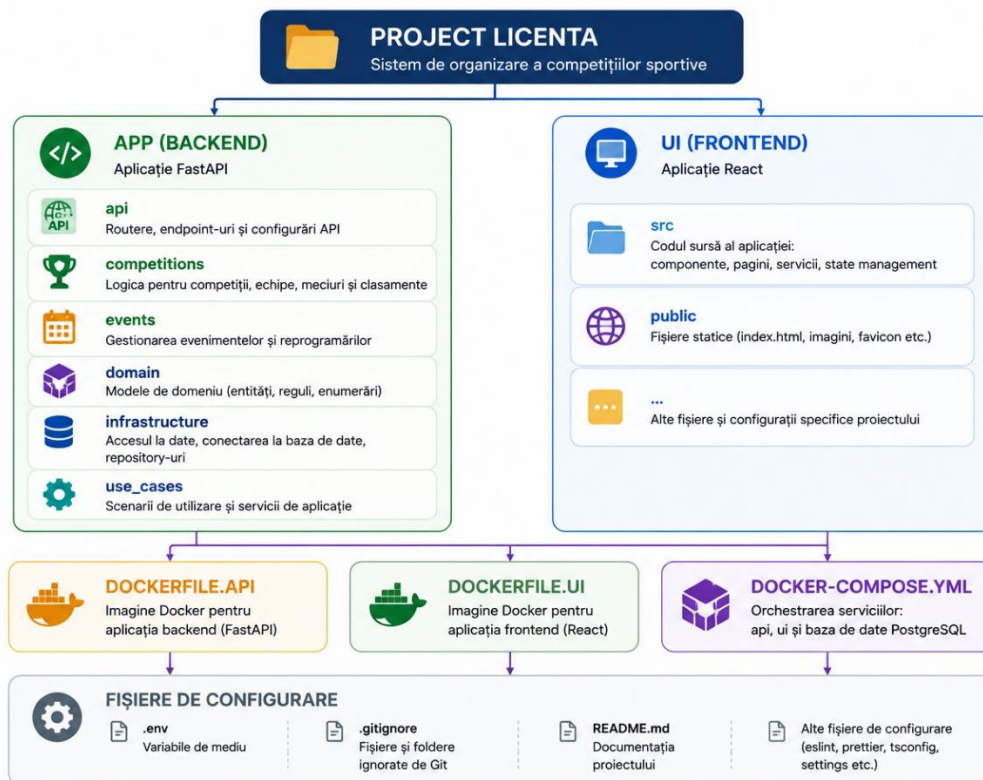


Figura 5.1 - Structura generală a proiectului

Figura 5.1 ilustrează organizarea aplicației în componente frontend și backend, precum și principalele module care contribuie la implementarea funcționalităților sistemului. De asemenea, sunt evidențiate fișierele utilizate pentru containerizare și configurare, care permit rularea și administrarea aplicației într-un mediu controlat.

5.2 IMPLEMENTAREA MODELULUI DE DOMENIU

5.2.1 ENTITATEA TEAM

Entitatea Team este utilizată pentru reprezentarea echipelor participante în competiție. Aceasta conține informațiile necesare identificării unei echipe și constituie unul dintre elementele fundamentale ale modelului de domeniu. Fiecare echipă este caracterizată printr-un identificator unic, un nume și orașul pe care îl reprezintă. Modelul este implementat utilizând mecanismul

dataclass din Python, care simplifică definirea entităților și gestionarea atributelor acestora.[15]

```
@dataclass(frozen=True)
```

```
class Team:
```

```
    name: str
```

```
    city: str = ""
```

```
    id: str = field(default_factory=lambda: uuid.uuid4().hex[:10])
```

Fragmentul de cod prezentat evidențiază implementarea entității Team. Utilizarea atributului frozen=True asigură caracterul imuabil al obiectelor după creare, contribuind la menținerea consistenței datelor în cadrul aplicației.

5.2.2 ENTITATEA PLAYER

Jucătorii participanți în competiție sunt modelați prin intermediul entității Player. Aceasta stochează informațiile esențiale necesare simulării sezonului și gestionării loturilor fiecărei echipe.

```
@dataclass(frozen=True)
```

```
class Player:
```

```
    id: str
```

```
    name: str
```

```
    pos: str
```

```
    height_cm: int
```

Entitatea Player permite reprezentarea fiecărui jucător prin intermediul unui identificator unic, al numelui, poziției ocupate în teren și al înălțimii. Aceste informații sunt utilizate în procesele de simulare și generare a statisticilor individuale.

5.2.3 ENTITATEA GAME

Entitatea Game reprezintă un meci desfășurat în cadrul unei competiții sportive și constituie una dintre cele mai importante structuri de date utilizate de aplicație. Aceasta conține informații despre echipele participante, locația desfășurării, intervalul de programare și rezultatul final al partidei.

```
@dataclass
```

```
class Game:
```

```
    stage: str = ""
```

```
    label: str = ""
```

```
    home: Team = None
```

```
    away: Team = None
```

```
    venue: Venue = None
```

```
    slot: Optional[TimeSlot] = None
```

```
    played: bool = False
```

```
    home_score: Optional[int] = None
```

```
    away_score: Optional[int] = None
```

Entitatea Game centralizează informațiile asociate unei partide și permite monitorizarea stării acesteia pe parcursul sezonului. Prin intermediul atributelor dedicate sunt gestionate echipele participante, programarea meciului și rezultatele generate în urma simulării.

5.3 IMPLEMENTAREA MECANISMULUI DE GENERARE A CALENDARULUI COMPETIȚIONAL

Mecanismul de generare a calendarului competițional reprezintă una dintre componentele centrale ale aplicației. Acesta este responsabil de crearea programului competițional și de alocarea meciurilor în intervale valide de desfășurare, respectând constrângerile definite pentru competiție.

Implementarea utilizează o combinație între algoritmul Round Robin pentru generarea meciurilor și un mecanism de planificare bazat pe constrângeri pentru alocarea acestora în calendar. Procesul urmărește identificarea unor intervale disponibile pentru desfășurarea partidelor și evitarea conflictelor asociate echipelor și locațiilor implicate.[17][11]

5.3.1 GENERAREA MECIURILOR

Pentru construirea sezonului regulat este utilizat algoritmul Round Robin, care generează toate confruntările necesare între echipele participante. Implementarea folosește metoda Circle Method, una dintre cele mai cunoscute tehnici pentru organizarea competițiilor de tip fiecare cu fiecare[17].

```
@staticmethod
```

```
def _round_robin_pairs(teams):
```

```
    n = len(teams)
```

```
    arr = teams[:]
```

```
    rounds = []
```

```
    for r in range(n - 1):
```

```
        pairs = []
```

```
        for i in range(n // 2):
```

```
a = arr[i]
b = arr[n - 1 - i]
pairs.append((a, b))

arr = [arr[0]] + [arr[-1]] + arr[1:-1]
rounds.append(pairs)

return rounds
```

Algoritmul generează etapele competiției prin rotirea succesivă a echipelor participante. În fiecare rundă sunt determinate perechile care vor disputa meciurile, iar după finalizarea unei etape pozițiile echipelor sunt actualizate pentru generarea următoarei runde. Această abordare asigură întâlnirea tuturor echipelor între ele pe parcursul sezonului.

5.3.2 ALOCAREA MECIURILOR ÎN CALENDAR

După generarea perechilor, sistemul începe procesul de planificare efectivă a meciurilor. Pentru fiecare partidă sunt analizate intervalele disponibile și locațiile care pot găzdui meciul respectiv.

```
for d in date_list:
    for t in time_list:
        slot = TimeSlot(
            date=d,
            start=t,
            duration_hours=self.match_duration_hours
        )

    for v in venue_candidates:
```

```
if self.is_resource_free(  
    slot,  
    v.id,  
    (game.home.id, game.away.id)  
):  
    game.slot = slot  
    game.venue = v  
    self.reserve(  
        slot,  
        v.id,  
        (game.home.id, game.away.id)  
    )  
    return True
```

Pentru fiecare meci sunt analizate succesiv datele disponibile, intervalele orare și locațiile candidate. În momentul identificării unui interval care respectă toate constrângerile definite, meciul este programat și resursele asociate sunt rezervate pentru a evita conflictele ulterioare.

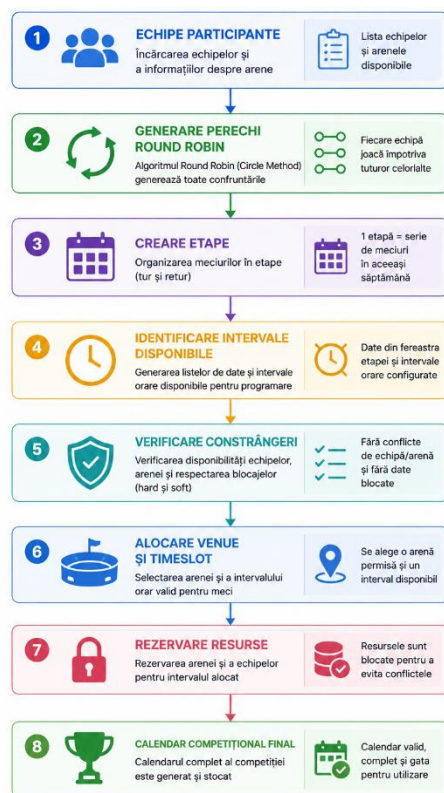


Figura 5.2 - Fluxul procesului de generare a calendarului competițional

Figura 5.2 prezintă principalele etape ale procesului de generare a calendarului competițional, de la generarea confruntărilor dintre echipe până la alocarea intervalelor de joc și obținerea calendarului final.

5.4 IMPLEMENTAREA MECANISMULUI DE SIMULARE A SEZONULUI

Simularea sezonului este coordonată prin funcția `simulate_season`, care reprezintă punctul central de orchestrare al aplicației. Aceasta încarcă datele inițiale, validează informațiile despre echipe și jucători, configurează constrângerile, inițializează scheduler-ul, pornește mecanismele de evenimente dinamice și rulează competițiile principale ale sezonului.

```

teams, venues, venues_by_team = load_teams_and_venues()
validate_teams_and_venues(teams, venues_by_team)
  
```

```
rosters_by_name = load_rosters_by_team_name()  
validate_rosters(rosters_by_name, teams)
```

```
sched = Scheduler(  
    teams=teams,  
    venues=venues,  
    constraints=constraints,  
    match_duration_hours=2,  
    time_slots=params.time_slots,  
    season_end=SEASON_END,  
    seed=params.seed,  
    event_log=log  
)
```

Acest fragment evidențiază etapa de inițializare a simulării. Sistemul încarcă echipele, sălile și loturile de jucători, apoi validează datele înainte de pornirea efectivă a sezonului. După această etapă este creat obiectul Scheduler, responsabil de programarea meciurilor și de respectarea constrângerilor definite.

Mai apoi, simularea parcurge principalele etape competiționale: Supercupa României, sezonul regulat, Cupa României și fazele de play-off și play-out.

```
supercup_game = sc.play(  
    champion=champion,  
    vice=vice,  
    cup_winner=prev_cup_winner,  
    venues_by_team=venues_by_team  
)
```

```
regular_games = rs.generate()
```

```
cup_winner, cup_games, _phases = run_cupa_romaniei(...)
```

```
po_games, final_order = run_playoff_and_playout(...)
```

Acest fragment arată modul în care simularea este organizată pe etape. Fiecare competiție este tratată separat, dar toate folosesc aceleași componente de planificare, simulare a rezultatelor și gestionare a datelor. La final, toate meciurile sunt reunite într-o singură listă, care reprezintă întregul sezon competițional generat de aplicație.

5.5 IMPLEMENTAREA MECANISMULUI DE REPROGRAMARE

Mecanismul de reprogramare are rolul de a modifica poziția unui meci în calendar atunci când acesta nu mai poate fi disputat conform planificării inițiale. O astfel de situație poate apărea în urma unui eveniment dinamic, a indisponibilității unei săli sau a apariției unui conflict de programare.[17]

În cadrul aplicației, reprogramarea este realizată prin funcția `reschedule_game_forward`, care caută un nou interval disponibil după data inițială a meciului. Înainte de căutarea unei alternative, vechea rezervare este eliminată temporar din calendar, pentru a permite verificarea corectă a resurselor disponibile.

```
old = game.slot
start_date = old.date + timedelta(days=1)

self.cal.unreserve(
    game.venue.id,
    (game.home.id, game.away.id),
    old
)

game.slot = None
```

Fragmentul prezentat evidențiază prima etapă a reprogramării. Sistemul reține intervalul inițial al meciului, stabilește o nouă dată de pornire pentru căutare și eliberează resursele ocupate anterior.

După eliminarea rezervării inițiale, sistemul încearcă să identifice un nou interval valid folosind aceeași logică de planificare utilizată la generarea calendarului.[11]

```
ok = self.schedule_game_in_window(  
    game,  
    (start_date, limit),  
    preferred_weekdays=preferred,  
    allow_alternate_venues=allow_alternate_venues,  
    randomize=True,  
    ignore_hard_blocks=False  
)
```

Dacă este identificat un interval valid, meciul este mutat automat, iar noua programare este înregistrată în jurnalul de evenimente. În cazul în care nu există o soluție validă, sistemul restaurează programarea inițială pentru a menține consistența calendarului.

5.6 IMPLEMENTAREA GESTIONĂRII EVENIMENTELOR DINAMICE

Pentru a simula situații apropiate de cele întâlnite în competițiile reale, aplicația include un mecanism de generare a evenimentelor dinamice. Acesta permite apariția unor situații neprevăzute care pot afecta desfășurarea sezonului și pot necesita modificarea calendarului competițional.[9]

Evenimentele sunt generate pe baza unor probabilități configurabile și includ situații precum indisponibilitatea unei săli, accidentarea unor jucători sau convocarea acestora la loturile naționale. În cazul în care un eveniment produce un conflict configurat de programare, sistemul inițiază procesul de reprogramare a meciurilor afectate.

```
p_covid_per_round: float = 0.05  
p_venue_block_per_round: float = 0.03  
p_concert_per_round: float = 0.18  
p_injury_per_round: float = 0.06  
p_callup_per_round: float = 0.12
```

Aceste probabilități permit generarea unor scenarii diferite la fiecare simulare și contribuie la evaluarea mecanismelor de planificare și reconfigurare implementate în cadrul aplicației.

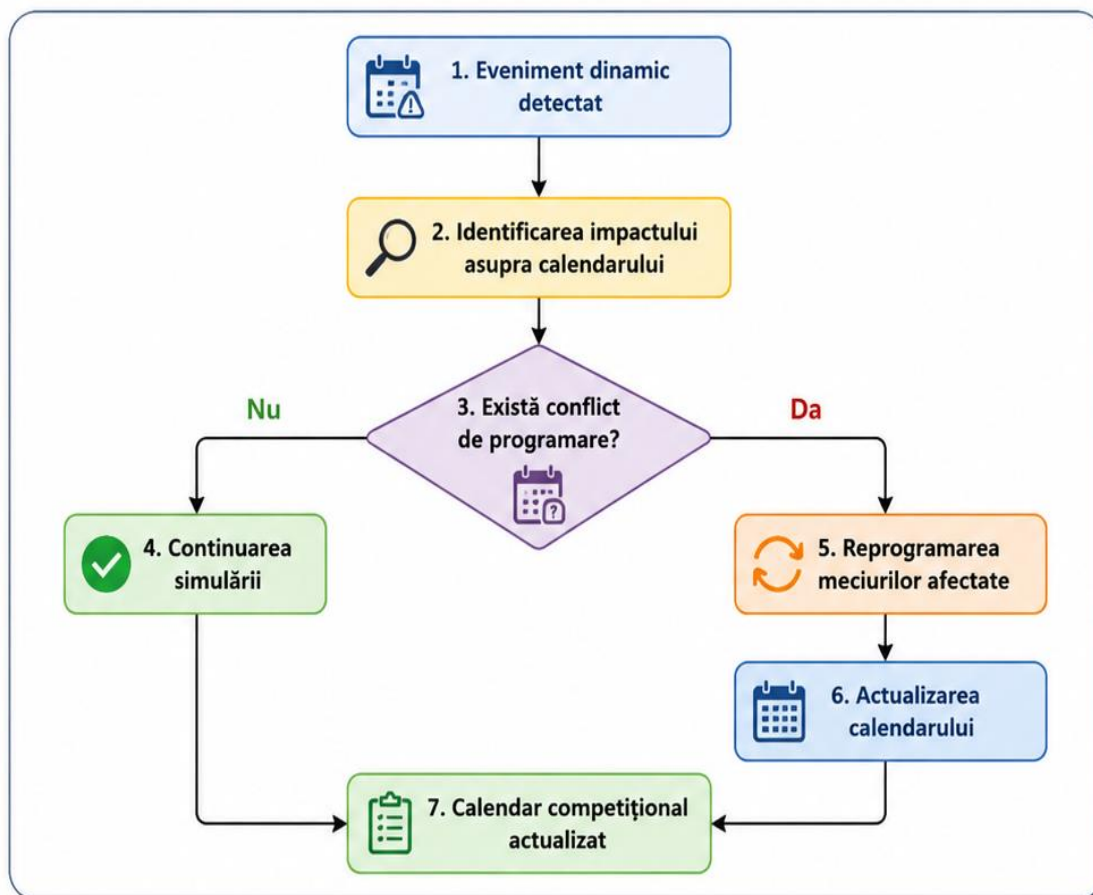


Figura 5.3 - Implementarea gestionării evenimentelor dinamice

Figura 5.3 ilustrează procesul de analiză și tratare a evenimentelor dinamice care pot afecta desfășurarea competiției.

5.7 IMPLEMENTAREA API-ULUI

Comunicarea dintre frontend și backend este realizată prin intermediul unei interfețe REST implementate utilizând framework-ul FastAPI. API-ul expune funcționalitățile principale ale aplicației și permite inițierea simulărilor, consultarea rezultatelor și exportarea datelor generate.[16][17]

Unul dintre cele mai importante endpoint-uri ale aplicației este cel utilizat pentru pornirea unei simulări.

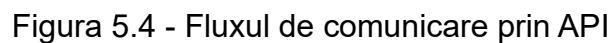
```
@app.post("/simulate")
def simulate(req: SimulateRequest):
    ...
    result = simulate_season(params)

    with SessionLocal() as session:
        run_id = save_run(session, result)

    return SimulateResponse(
        run_id=run_id,
        message="Simulation completed",
        seed=seed
    )
```

Acest endpoint primește parametrii configurați de utilizator, execută simularea sezonului și salvează rezultatele obținute în baza de date. La final este returnat identificatorul simulării, care poate fi utilizat ulterior pentru accesarea informațiilor generate.

Pe lângă funcționalitatea de simulare, API-ul oferă endpoint-uri pentru consultarea meciurilor, evenimentelor, statisticilor și pentru exportarea rezultatelor în diferite formate.



5.8 IMPLEMENTAREA INTERFEȚEI UTILIZATOR

În cadrul paginii Runs, utilizatorul poate vizualiza toate simulările salvate în sistem, fiecare rulare fiind identificată printr-un identificator unic și însoțită de informații relevante precum seed-ul utilizat data generării și rezultatele

finale. Acolo, apăsând pe butonul Open al unei simulări, acesta poate vizualiza calendarul meciurilor, clasamentele, statisticile jucătorilor și jurnalul evenimentelor. Toate aceste date pot fi și exportate dacă utilizatorul își dorește acest lucru, apăsând pe butonul Download și fișierul dorit (de exemplu, Download games.csv).

Pentru administrarea eficientă a rulărilor, interfața oferă și funcționalitatea Deleter runs, care permite ștergerea completă a tuturor simulărilor existente. Această operație elimină atât rulările principale, cât și datele asociate acestora (meciuri, rezultate, clasamente finale), resetând totodată contorul intern al rulărilor. Astfel, utilizatorul poate relua procesul de simulare de la o stare inițială curată.

Simulate season

Seed (optional) 123

TV featured per round 1

TV requested time (optional) HH:MM (e.g. 20:00)

Time slots (CSV) 17:00,17:15,17:30,17:45,18:00,18:15,18:30,18:45,19:00,19:15,19:30,19:45,20:00,20:15,20:30,20:45,21:00

TV allowed times (CSV, optional) e.g. 19:30,20:00,20:30

Dynamic events probabilities

p_covid_per_round 0.05

p_venue_block_per_round 0.03

p_concert_per_round 0.18

p_injury_per_round 0.06

p_callup_per_round 0.12

p_covid_player_outbreak_per_round 0.04

Run simulation

Figura 5.5 - Interfața de configurare a simulării

Simulate **Runs**

Delete runs

Run #1 — seed: 9403632195 [Open](#)
created_at: 2026-06-17 21:59:42
Champion: **CSO Voluntari** | Vice: **U-BT Cluj-Napoca**
Cup: **CSM CSU Oradea** | Supercup: **U-BT Cluj-Napoca**

Figura 5.6 - Vizualizarea rulărilor efectuate

[Simulate](#) [Runs](#)

Run #1

Seed: 123
Created: 2026-06-08 18:38:02
Champion: U-BT Cluj-Napoca | **Vice:** CSM CSU Oradea
Cup: SCM Politehnica Timisoara | **Supercup:** U-BT Cluj-Napoca
Games count: 333

[Download games.csv](#) [Download players_avg.csv](#) [Download events.txt](#) [Download regular_standings.csv](#)
[Download final_order.csv](#)

[Games](#) [Players avg](#) [Events](#)

SUPERCUP — SUPERCUPA ROMANIEI
2026-09-12 20:15 • U-BT Cluj-Napoca 83-79 CSM CSU Oradea • BT Arena (Cluj-Napoca)
TV: false | requested: - | confirmed: false
Notes: Cup winner = champion -> vs vice

CUP Q — CUP 1/16 TIE3 L1
2026-09-21 19:30 • Dinamo Bucuresti 90-81 CSM Galati • Sala Dinamo (Bucuresti)
TV: false | requested: - | confirmed: false

Figura 5.7 - Vizualizarea rezultatelor unei simulări

6. TESTAREA ȘI VALIDAREA SISTEMULUI

6.1 METODOLOGIA DE TESTARE

Testarea sistemului a fost realizată atât în mediul local, prin rularea infrastructurii complete utilizând Docker Compose, cât și în mediul online, prin intermediul versiunii deployate în cloud folosind platforma Railway. În ambele medii au fost executate simulări complete ale sezonului competițional și au fost analizate rezultatele generale. Scopul testării a fost verificarea funcționării corecte a mecanismelor de programare a meciurilor, simulare a rezultatelor, gestionare a evenimentelor dinamice și reprogramare a partidelor afectate de conflicte.

Procesul de validare a urmărit atât corectitudinea funcțională a aplicației, cât și consistența datelor generate pe parcursul simulării. Pentru fiecare rulare au fost analizate calendarul competițional, clasamentele rezultate, statisticile jucătorilor și jurnalul evenimentelor produse.

Rezultatele obținute au fost exportate în formatele CSV și TXT și au fost utilizate pentru verificarea comportamentului sistemului în diferite scenarii de utilizare.

6.2 VALIDAREA GENERĂRII CALENDARULUI

Calendarul competițional generat de aplicație a fost analizat pentru verificarea programării corecte a meciurilor. Rezultatele obținute confirmă faptul că sistemul a reușit să programeze partidele sezonului regulat și ale competițiilor eliminatorii, respectând intervalele orare și locațiile disponibile.

	stage	label	date	time	home	away	home_score	away_score	venue	tv_featured	tv_requested_time	tv_confirmed	notes
1	SUPERCUP	SUPERCUPA ROMANIEI	03/09/2026	21:00	U-BT Cluj-Napoca	CSM CSU Oradea	75	72	BT Arena (Cluj-Napoca)	FALSE		FALSE	Cup winner = champion -> vs vice
2	CUP_Q	CUP 1/16 TIE4 L1	20/09/2026	20:30	CSM Galati	Dinamo Bucuresti	81	80	Sala Sporturilor Dunarea (Galati)	FALSE		FALSE	
3	CUP_Q	CUP 1/16 TIE2 L1	22/09/2026	19:00	CSM Targu Mures	CSM Targu Jiu	69	63	Sala Sporturilor Targu Mures (Targu Mures)	FALSE		FALSE	
4	CUP_Q	CUP 1/16 TIE1 L1	22/09/2026	20:00	CSM BBA Petrolul Ploiesti	CSA Steaua Sharks Bucuresti	76	75	Sala Olimpia (Ploiesti)	FALSE		FALSE	
5	CUP_Q	CUP 1/16 TIE3 L1	24/09/2026	17:30	Rapid Bucuresti	CSU Sibiu	86	77	Sala Rapid (Bucuresti)	FALSE		FALSE	
6	CUP_Q	CUP 1/16 TIE3 L2	26/09/2026	17:45	CSU Sibiu	Rapid Bucuresti	65	62	Sala Transilvania (Sibiu)	FALSE		FALSE	
7	CUP_Q	CUP 1/16 TIE2 L2	27/09/2026	18:00	CSM Targu Jiu	CSM Targu Mures	68	77	Sala Sporturilor Targu Jiu (Targu Jiu)	FALSE		FALSE	
8	CUP_Q	CUP 1/16 TIE3 L2	27/09/2026	19:00	CSA Steaua Sharks Bucuresti	CSM BBA Petrolul Ploiesti	83	72	Sala Polivalenta Steaua (Bucuresti)	FALSE		FALSE	
9	CUP_Q	CUP 1/16 TIE4 L2	27/09/2026	20:00	Dinamo Bucuresti	CSM Galati	83	62	Sala Dinamo (Bucuresti)	FALSE		FALSE	
10	REGULAR	Etapa 1	02/10/2026	21:00	CSM BBA Petrolul Ploiesti	SCM Politehnica Timisoara	83	93	Sala Olimpia (Ploiesti)	FALSE		FALSE	
11	REGULAR	Etapa 1	02/10/2026	21:00	CSM Galati	Dinamo Bucuresti	85	88	Sala Sporturilor Dunarea (Galati)	FALSE		FALSE	
12	REGULAR	Etapa 1	02/10/2026	21:00	CSM Targu Jiu	CSU Sibiu	84	93	Sala Sporturilor Targu Jiu (Targu Jiu)	FALSE		FALSE	
13	REGULAR	Etapa 1	04/10/2026	17:45	CS Valcea 1924	U-BT Cluj-Napoca	81	80	Sala Traian (Ramnicu Valcea)	FALSE		FALSE	
14	REGULAR	Etapa 1	04/10/2026	17:45	CSM Corona Brasov	FC Arges Pitesti	87	88	Sala Sporturilor Dumitru Popescu Colibasi (Brasov)	FALSE		FALSE	
15	REGULAR	Etapa 1	04/10/2026	19:00	CSA Steaua Sharks Bucuresti	SCMU Craiova	116	121	Sala Polivalenta Steaua (Bucuresti)	TRUE	19:00	TRUE	
16	REGULAR	Etapa 1	04/10/2026	19:00	CSM Targu Mures	CSO Voluntari	83	90	Sala Sporturilor Targu Mures (Targu Mures)	FALSE		FALSE	
17	REGULAR	Etapa 1	04/10/2026	20:00	CSM CSU Oradea	Rapid Bucuresti	58	55	Oradea Arena (Oradea)	FALSE		FALSE	
18	REGULAR	Etapa 2	10/10/2026	18:15	CSM Galati	CSO Voluntari	83	103	Sala Sporturilor Dunarea (Galati)	TRUE	18:15	TRUE	
19	REGULAR	Etapa 2	10/10/2026	19:30	CSA Steaua Sharks Bucuresti	Rapid Bucuresti	85	87	Sala Polivalenta Steaua (Bucuresti)	FALSE		FALSE	
20	REGULAR	Etapa 2	10/10/2026	21:00	CSM Targu Jiu	CSM Targu Mures	60	68	Sala Sporturilor Targu Jiu (Targu Jiu)	FALSE		FALSE	
21	REGULAR	Etapa 2	11/10/2026	18:45	CS Valcea 1924	SCMU Craiova	100	92	Sala Traian (Ramnicu Valcea)	FALSE		FALSE	
22	REGULAR	Etapa 2	11/10/2026	19:15	CSM CSU Oradea	Dinamo Bucuresti	72	55	Oradea Arena (Oradea)	FALSE		FALSE	
23	REGULAR	Etapa 2	11/10/2026	20:15	CSM BBA Petrolul Ploiesti	FC Arges Pitesti	71	74	Sala Olimpia (Ploiesti)	FALSE		FALSE	
24	REGULAR	Etapa 2	11/10/2026	20:30	U-BT Cluj-Napoca	SCM Politehnica Timisoara	93	81	BT Arena (Cluj-Napoca)	FALSE		FALSE	

Tabelul 1 - Fragment din calendarul competițional generat de sistem

Calendarul conține informații referitoare la competiție, etapă, data și ora desfășurării meciurilor, echipele participante și locația de joc. Analiza rezultatelor confirmă funcționarea corectă a mecanismului de planificare implementat.

6.3 VALIDAREA SIMULĂRII SEZONULUI

Simularea sezonului a generat automat clasamentul final al sezonului regulat pe baza rezultatelor obținute în meciurile disputate. Clasamentul este construit utilizând numărul de puncte acumulate, numărul de victorii și înfrângeri, precum și diferența dintre punctele marcate și cele primite.

Rezultatele obținute evidențiază o distribuție realistă a performanțelor echipelor participante. În simularea analizată, clasamentul reflectă evoluția echipelor pe parcursul întregului sezon și permite identificarea formațiilor calificate în fazele eliminatorii.

Poz.	Echipa	Pct	V	I	+/-
1	U-BT Cluj-Napoca	56	26	4	254
2	CSO Voluntari	56	26	4	242
3	CSM CSU Oradea	55	25	5	221
4	CS Valcea 1924	54	24	6	221
5	SCM Politehnica Timisoara	51	21	9	110
6	FC Arges Pitesti	51	21	9	109
7	CSM Corona Brasov	48	18	12	79
8	Rapid Bucuresti	47	17	13	30
9	SCMU Craiova	46	16	14	54
10	CSM Targu Mures	42	12	18	-49
11	CSU Sibiu	40	10	20	-84
12	Dinamo Bucuresti	38	8	22	-139
13	CSM BBA Petrolul Ploiesti	37	7	23	-220
14	CSA Steaua Sharks Bucuresti	36	6	24	-157
15	CSM Targu Jiu	32	2	28	-324
16	CSM Galati	31	1	29	-347

Tabelul 2 - Clasamentul final al sezonului regulat

Tabelul 2 prezintă clasamentul sezonului regulat rezultat în urma simulării. Pozițiile echipelor au fost determinate pe baza punctelor acumulate și a rezultatelor obținute pe parcursul competiției.

6.4 VALIDAREA MECANISMULUI DE REPROGRAMARE

Funcționarea mecanismului de programare a fost verificată prin analiza jurnalelor generate în timpul simulării. Log-urile conțin informații despre meciurile programate și modificările efectuate asupra calendarului. Rezultatele confirmă înregistrarea corectă a operațiilor realizate de sistem.

```
[2026-06-08 18:37:47] SCHEDULE: SUPERCUP SUPERCUPA ROMANIEI: U-BT Cluj-Napoca vs CSM CSU Oradea -> 2026-09-03 21:00 @ BT Arena (Cluj-Napoca) (game_id=9c7405d77c)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 1: CS Valcea 1924 vs U-BT Cluj-Napoca -> 2026-10-04 17:45 @ Sala Traian (Ramnicu Valcea) (game_id=cd108affc1)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 1: CSA Steaua Sharks Bucuresti vs SCMU Craiova -> 2026-10-04 18:30 @ Sala Polivalenta Steaua (Bucuresti) (game_id=14c195e17e)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 1: CSM BBA Petrolul Ploiesti vs SCM Politehnica Timisoara -> 2026-10-02 21:00 @ Sala Olimpia (Ploiesti) (game_id=3e840f69ed)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 1: CSM CSU Oradea vs Rapid Bucuresti -> 2026-10-04 20:00 @ Oradea Arena (Oradea) (game_id=566b2289a0)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 1: CSM Corona Brasov vs FC Arges Pitesti -> 2026-10-04 17:45 @ Sala Sporturilor Dumitru Popescu Colibasi (Brasov) (game_id=a2401450a2)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 1: CSM Galati vs Dinamo Bucuresti -> 2026-10-02 21:00 @ Sala Sporturilor Dunarea (Galati) (game_id=71c2a9a340)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 1: CSM Targu Jiu vs CSU Sibiu -> 2026-10-02 21:00 @ Sala Sporturilor Targu Jiu (Targu Jiu) (game_id=52e975f37d)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 1: CSM Targu Mures vs CSO Voluntari -> 2026-10-04 19:00 @ Sala Sporturilor Targu Mures (Targu Mures) (game_id=2445155220)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 2: CS Valcea 1924 vs SCMU Craiova -> 2026-10-11 18:45 @ Sala Traian (Ramnicu Valcea) (game_id=1abde95c1f)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 2: U-BT Cluj-Napoca vs SCM Politehnica Timisoara -> 2026-10-11 20:30 @ BT Arena (Cluj-Napoca) (game_id=c23845fb34)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 2: CSA Steaua Sharks Bucuresti vs Rapid Bucuresti -> 2026-10-10 19:30 @ Sala Polivalenta Steaua (Bucuresti) (game_id=7ea62bf6c6)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 2: CSM BBA Petrolul Ploiesti vs FC Arges Pitesti -> 2026-10-11 20:15 @ Sala Olimpia (Ploiesti) (game_id=4abf8142de)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 2: CSM CSU Oradea vs Dinamo Bucuresti -> 2026-10-11 19:15 @ Oradea Arena (Oradea) (game_id=1b44f7f209)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 2: CSM Corona Brasov vs CSU Sibiu -> 2026-10-11 20:45 @ Sala Sporturilor Dumitru Popescu Colibasi (Brasov) (game_id=cee5ec7f06)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 2: CSM Galati vs CSO Voluntari -> 2026-10-10 18:15 @ Sala Sporturilor Dunarea (Galati) (game_id=9e0073ee42)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 2: CSM Targu Jiu vs CSM Targu Mures -> 2026-10-10 21:00 @ Sala Sporturilor Targu Jiu (Targu Jiu) (game_id=ee5dda55f9)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 3: CS Valcea 1924 vs SCM Politehnica Timisoara -> 2026-10-18 18:00 @ Sala Traian (Ramnicu Valcea) (game_id=c9ee83c3a4)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 3: SCMU Craiova vs Rapid Bucuresti -> 2026-10-17 17:00 @ Sala Polivalenta Craiova (Craiova) (game_id=23e6487daa)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 3: U-BT Cluj-Napoca vs FC Arges Pitesti -> 2026-10-17 18:45 @ BT Arena (Cluj-Napoca) (game_id=7f7134a1ac)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 3: CSA Steaua Sharks Bucuresti vs Dinamo Bucuresti -> 2026-10-17 20:15 @ Sala Polivalenta Steaua (Bucuresti) (game_id=67ed53de77)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 3: CSM BBA Petrolul Ploiesti vs CSU Sibiu -> 2026-10-17 19:15 @ Sala Olimpia (Ploiesti) (game_id=e6d0e76ae0)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 3: CSM CSU Oradea vs CSO Voluntari -> 2026-10-18 18:30 @ Oradea Arena (Oradea) (game_id=4449515ccb)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 3: CSM Corona Brasov vs CSM Targu Mures -> 2026-10-17 20:00 @ Sala Sporturilor Dumitru Popescu Colibasi (Brasov) (game_id=22c7ec0932)
[2026-06-08 18:37:47] SCHEDULE: REGULAR Etapa 3: CSM Galati vs CSM Targu Jiu -> 2026-10-17 19:15 @ Sala Sporturilor Dunarea (Galati) (game_id=c7997741d1)
```

Figura 6.1 - Log-uri

6.5 ANALIZA REZULTATELOR OBȚINUTE

Rezultatele obținute confirmă funcționarea corectă a sistemului dezvoltat. Aplicația generează automat calendarul competițional, simulează desfășurarea sezonului și produce clasamente și rapoarte exportabile. Datele generate sunt consistente și respectă constrângerile definite în procesul de planificare.

7. CONCLUZII ȘI DIRECȚII VIITOARE DE DEZVOLTARE

În cadrul prezentei lucrări a fost proiectată și implementată o aplicație destinată simulării și gestionării unui sezon competițional de baschet. Soluția dezvoltată permite generarea automată a calendarului competițional, simularea rezultatelor meciurilor și administrarea principalelor competiții din cadrul unui sezon sportiv.

Obiectivele stabilite la începutul lucrării au fost îndeplinite prin realizarea unei arhitecturi moderne bazate pe separarea componentelor frontend și backend, utilizarea unei baze de date relaționale și implementarea unor mecanisme automate pentru programarea și simularea competițiilor. Sistemul oferă posibilitatea configurării parametrilor de simulare și generării unor rapoarte complete privind desfășurarea sezonului.

Rezultatele obținute în urma testării demonstrează funcționarea corectă a aplicației și capacitatea acesteia de a genera calendare, clasamente și statistici într-un mod automatizat. Totodată, utilizarea evenimentelor dinamice contribuie la creșterea realismului simulării și permite evaluarea impactului unor situații neprevăzute asupra desfășurării competiției.

Ca direcții viitoare de dezvoltare, aplicația poate fi extinsă prin integrarea unor modele statistice mai avansate pentru simularea meciurilor, utilizarea unor surse reale de date sportive și dezvoltarea unor funcționalități suplimentare pentru analiza performanțelor echipelor și jucătorilor. De asemenea, poate fi luat în calcul implementarea unor module dedicate managementului financiar, transferurilor de jucători sau simulării mai multor sezoane consecutive.

În forma actuală, aplicația este disponibilă atât într-o variantă locală destinată dezvoltării și testării, cât și într-o variantă online complet funcțională, accesibilă public prin intermediul browser-ului. Acest lucru permite utilizarea și demonstrarea sistemului în condiții reale, validând atât arhitectura software propusă, cât și mecanismele de planificare și reconfigurare implementate.

DECLARAȚIE PRIVIND UTILIZAREA INSTRUMENTELOR DE INTELIGENȚĂ ARTIFICIALĂ

Doresc să recunosc utilizarea ChatGPT în redactarea acestei lucrări. Acesta a fost utilizat în principiu pentru îmbunătățirea clarității exprimării și pentru asigurarea unei structuri coerente a textului, dar și pentru o mai bună construcție a imaginilor, diagramelor prezente în lucrare datorită gradului avansat al acestora la nivel de detaliu și claritate.

LISTA FIGURILOR

Figura 2.1 – Structura sezonului competițional utilizat în aplicație, adaptare proprie după regulamentul FRB[8].....	- 14 -
Figura 2.2 - Model general al unei probleme de planificare (scheduling).....	- 16 -
Figura 2.3 - Principalele categorii de constrângeri utilizate în planificare.....	- 18 -
Figura 2.4 - Procesul general de reprogramare a unui meci	- 20 -
Figura 2.5 - Rolul simulării în cadrul sistemului	- 22 -
Figura 3.1 - Funcționalitățile principale ale sistemului	- 25 -
Figura 3.2 - Interacțiunea utilizatorului cu sistemul	- 28 -
Figura 3.3 - Principalele cazuri de utilizare ale sistemului	- 30 -
Figura 3.4 - Arhitectura conceptuală a sistemului.....	- 32 -
Figura 4.1 - Arhitectura software a aplicației	- 37 -
Figura 4.2 - Structura componentei frontend.....	- 40 -
Figura 4.3 - Structura componentei backend	- 42 -
Figura 4.4 - Modelul de domeniu al aplicației.....	- 44 -
Figura 4.5 - Structura datelor gestionate de aplicație	- 46 -
Figura 4.6 - Arhitectura containerizată a aplicației.....	- 48 -
Figura 4.7 - Implementarea și orchestrarea componentelor aplicației în mediul cloud utilizând Railway (frontend, backend și baza de date PostgreSQL)	- 50 -
Figura 5.1 - Structura generală a proiectului	- 52 -
Figura 5.2 - Fluxul procesului de generare a calendarului competițional	- 58 -
Figura 5.3 - Implementarea gestionării evenimentelor dinamice	- 62 -
Figura 5.4 - Fluxul de comunicare prin API.....	- 64 -
Figura 5.5 - Interfața de configurare a simulării	- 65 -
Figura 5.6 - Vizualizarea rulărilor efectuate	- 65 -
Figura 5.7 - Vizualizarea rezultatelor unei simulări.....	- 66 -
Figura 6.1 - Log-uri	- 70 -

LISTA TABELELOR

Tabelul 1 - Fragment din calendarul competițional generat de sistem	- 68 -
Tabelul 2 - Clasamentul final al sezonului regulat	- 69 -

BIBLIOGRAFIE

- [1] Alembic, Alembic Documentation, disponibil la:
<https://alembic.sqlalchemy.org>, accesat la data de 2026/06/06.
- [2] Apache Software Foundation, Apache ECharts Documentation, disponibil la: <https://echarts.apache.org>, accesat la data de 2026/06/06.
- [3] Baptiste, P., Le Pape, C. și Nuijten, W., Constraint-Based Scheduling, Springer, 2001.
- [4] Cormen, T. H., Leiserson, C. E., Rivest, R. L. și Stein, C., Introduction to Algorithms, 4th ed., MIT Press, 2022.
- [5] Docker Inc., Docker Documentation, disponibil la:
<https://docs.docker.com>, accesat la data de 2026/07/06.
- [6] FastAPI, FastAPI Documentation, disponibil la:
<https://fastapi.tiangolo.com>, accesat la data de 2026/07/06.
- [7] Fédération Internationale de Basketball (FIBA), Official Basketball Rules, disponibil la: <https://www.fiba.basketball>, accesat la data de 2026/06/06
- [8] Federația Română de Baschet (FRB), Regulamente oficiale și structura competițională, disponibil la: <https://www.frbaschet.ro>, accesat la data de 2026/06/06.
- [9] Fowler, M., Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.
- [10] Gamma, E., Helm, R., Johnson, R. și Vlissides, J., Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.
- [11] Google OR-Tools, Optimization Tools Documentation, disponibil la:
<https://developers.google.com/optimization>, accesat la data de 2026/07/06.
- [12] Martin, R. C., Clean Architecture: A Craftsman's Guide to Software Structure and Design, Pearson Education, 2017.

- [13] Pinedo, M., Scheduling: Theory, Algorithms, and Systems, 5th ed., Springer, 2016.
- [14] PostgreSQL Global Development Group, PostgreSQL Documentation, disponibil la: <https://www.postgresql.org/docs>, accesat la data de 2026/10/06.
- [15] Pydantic, Pydantic Documentation, disponibil la: <https://docs.pydantic.dev>, accesat la data de 2026/09/06.
- [16] Richardson, T. și Ruby, S., RESTful Web Services, O'Reilly Media, 2007.
- [17] SQLAlchemy, SQLAlchemy Documentation, disponibil la: <https://docs.sqlalchemy.org>, accesat la data de 2026/10/06.